

CSS基础知识点复习

CSS知识点复习, 本文档整体基于MDN进行整理。若有不正确之处以官方文档为标准, 以下为几点说明:

- 列举属性时并未列出所有属性, 而是选择了其中比较常用、实用的属性。
- 第一部分列举出了一些面试题和小案例, 第二部分主要是知识点总结, 文档有目录结构, 可配合使用。
- 本文档以CSS属性应用和基础概念为主, 不过多涉及如浏览器渲染过程, 性能优化等。

0. 面试题与案例

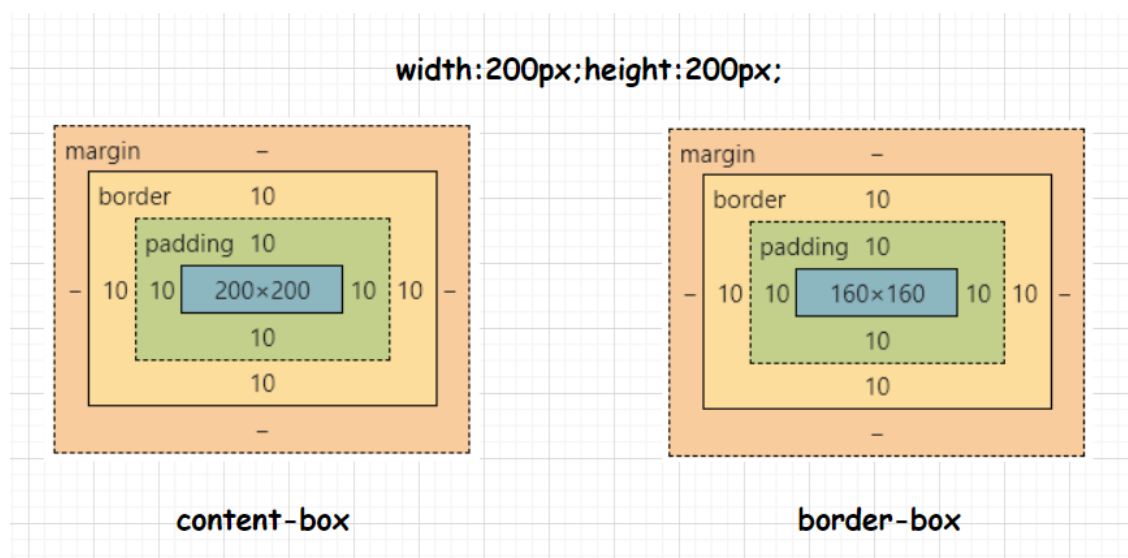
0.1 盒子

0.1.1 介绍一下CSS标准盒模型和IE盒模型?

CSS中组成一个块级盒子需要:

- **Content box:** 这个区域是用来显示内容, 大小可以通过设置 `width` 和 `height`。
- **Padding box:** 包围在内容区域外部的空白区域; 大小通过 `padding` 相关属性设置。
- **Border box:** 边框盒包裹内容和内边距。大小通过 `border` 相关属性设置。
- **Margin box:** 这是最外面的区域, 是盒子和其他元素之间的空白区域。大小通过 `margin` 相关属性设置。

标准盒模型中设置宽高实际上设置的是**Content box**, 而IE盒模型设置的是**Border box**, 我们可以通过设置属性 `box-sizing` 来控制盒模型的表现。



0.1.2 介绍一下BFC?

[BFC块格式化上下文](#)

0.1.3 隐藏元素的方法和区别

- `display: none` 渲染时不会包含该元素。
- `visibility: hidden` 该元素在页面中仍占据空间, **不会响应**绑定的监听事件。
- `opacity: 0` 设置透明度为0, 元素在页面中仍然占据空间, **能够响应**绑定的监听事件。
- 通过 `transform/position/margin`: 将元素移除可视区域。

- `transform: scale(0)`: 将元素缩小到0。

0.1.3 文本溢出隐藏

单行文本

```
.outer {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

多行文本

```
.outer {
  overflow: hidden;
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: xxx; /* xxx为限制多少行文本 */
}
```

0.1.3 实现长宽比保持不变的盒子

此处以实现一个正方形为例

传统方式

```
.outer {
  position: relative;
  width: 15%;
  padding-top: 15%;
}

.inner {
  position: absolute;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
}
```

对比



aspect-ratio

`aspect-ratio` 为box容器规定了一个期待的**纵横比**，这个纵横比可以用来计算自动尺寸以及为其他布局函数服务。

```
.outer {  
  width: 15%;  
}  
  
.inner {  
  width: 100%;  
  aspect-ratio: 1 / 1;  
}
```

0.2 布局

0.2.1 实现水平居中的多种方式

行内元素

`text-align` 并不控制块元素自己的对齐，只控制它的行内内容的对齐。

```
.outer {  
  text-align: center;  
}
```

块级元素

```
.inner {  
  width: xxxpx;  
  margin: 0 auto;  
}
```

通用方法

1. `margin: 0 auto`

```
.inner {  
    /* 这里使用max-content 和min-content也行 */  
    width: fit-content; /* 需要注意的是该写法目前在一些低版本浏览器和IE不支持 */  
}
```

2. `position`

```
.inner {  
    position: absolute;  
    left: 50%;  
    /* 已知宽度 */ margin-left: -50px;  
    /* 未知宽度 */ transform: translateX(-50%);  
}
```

下面这样也是可以的(宽度已知)

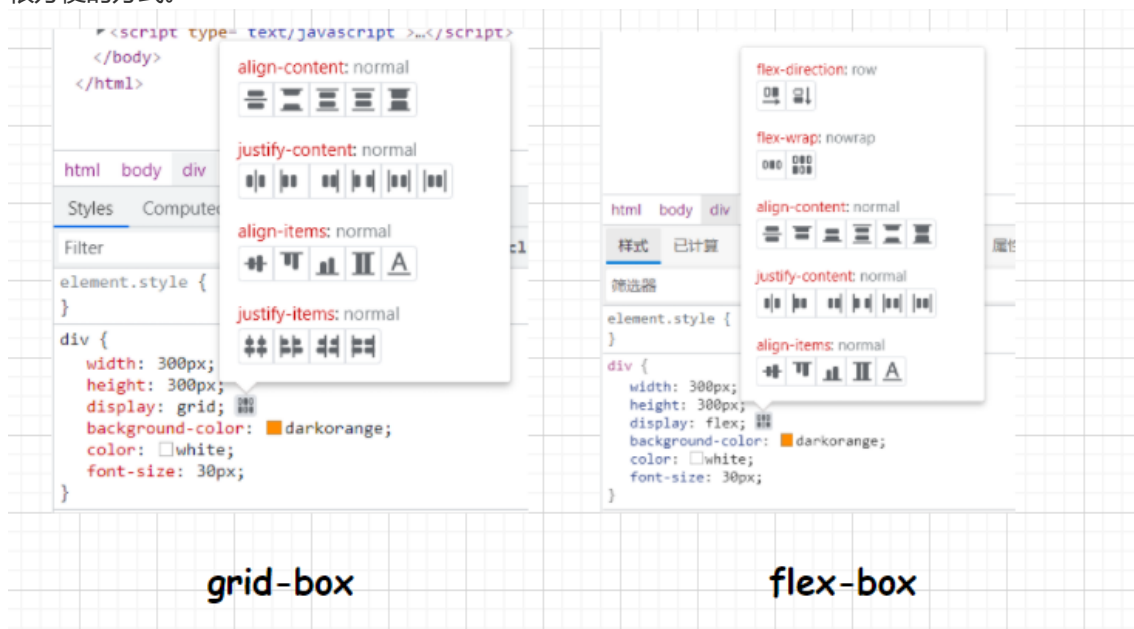
```
.inner {  
    position: absolute;  
    left: calc(50% - 50px);  
    right: calc(50% - 50px);  
    /* 这样指定相当于设定了width为100px 若指定宽度属性的话就不是居中的了 */  
}
```

3. 弹性盒子 flex

```
.outer {  
    display: flex;  
    justify-content: center; /* 主轴居中 */  
}
```

4. 栅格布局 grid

在 grid 中也是可以实用 `justify-content` 等对齐方式的，此处不再列举，浏览器为我们提供了很方便的方式。



这里我们使用 `grid` 特有的方式

```
.outer {
  display: grid;
  /* grid-template: auto; 这种方法可以了解一下grid的自动布局*/
  grid-template: 1fr auto 1fr / 1fr auto 1fr; /* 显式创建了3x3的栅格*/
  /* 此处的auto可以自己替换成max-content, fit-content等属性试试 */
}
.inner {
  grid-area: 2 / 2;
}
```

上面是比较常见的组合，还可以有更多写法，比如在宽度已知的情况下还可以 `margin + calc` / `transform`, `position + calc` 等，`fit-content` 等属性也是非常好用的。

0.2.2 实现垂直居中的多种方式

行内元素

```
.outer {
  line-height: xxxpx;
}
/* 当外层元素的高度与line-height的时候垂直居中，如果不设置height的话line-height就是元素的最小高度，height与其一致。*/
```

块级元素

```
.outer {
  padding: xxxpx xxxpx;
}
```

或外层元素自适应高度的情况下（固定的情况可以通过计算）

```
.innter {
  margin: xxxpx auto;
}
```

通用方法

1. 外层元素上下padding相同

```
.outer {
  padding: xxxpx xxxpx;
}
```

2. `vertical-align: middle`

```
.outer {
  display: table-cell; /* display需要是table-cell */
  vertical-align: middle;
}
```

3. `position`, `flex`, `grid` 等写法与水平类似，换成垂直方向即可。

0.2.3 经典布局

本节介绍一些经典布局如圣杯，双飞翼布局等，实现方法远多于下面列举的。

两列布局

特征：一端宽度固定，另一端自适应。



方法一： `float` + `margin-left`

```
.outer {
  position: absolute;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  background-color: #f4f6fa;
}

.inner-left {
  width: 250px;
  float: left;
  height: 100%;
  background-color: #353b48;
}

.inner-right {
  margin-left: 250px;
  background-color: #fff;
  height: 100%;
}
```

方法二： `float` + `clear: right`

outer 和 inner-left 部分不变，修改 inner-right

```
.inner-right {
  clear: right;
  padding-left: 250px;
  background-color: #fff;
  height: 100%;
}
```

可以尝试用创建 **BFC** 的方法代替比如给 inner-right 一个 `overflow: hidden`, `display: flow-root` 等

方法三: `calc()`

左边固定, 右边通过 `calc()` 函数计算

```
.inner-right {
  float: left;
  width: calc(100% - 250px);
  background-color: #fff;
  height: 100%;
}
```

方法四: 弹性盒子

这个方法写起来简单的多, 相信也是大家现在用的比较多的, 给父元素增加一个 `display: flex`

```
.outer {
  position: absolute;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  display: flex;
  background-color: #f4f6fa;
}

.inner-left {
  width: 250px;
  background-color: #353b48;
}

.inner-right {
  flex: 1;
  background-color: #fff;
}
```

还有例如**栅格布局**和**column**布局等方法, 这里不再列举。

三列布局

左右两侧固定, 中间自适应

以下代码中删除了一些非核心部分的代码如颜色字号等



方法一：flex 先来个最简单的方法体验一下

```
<style>
  .outer {
    position: absolute;
    top: 80px;
    right: 0;
    bottom: 80px;
    left: 0;
    display: flex;
  }

  .inner-left {
    width: 250px;
  }

  .inner-right {
    width: 250px;
  }

  .inner-center {
    flex: 1;
  }

  header,
  footer {
    position: absolute;
    left: 0;
    z-index: 2;
    width: 100%;
    line-height: 80px;
  }

  header {
    top: 0;
  }

  footer {
    bottom: 0;
  }
</style>
```



```

    }
</style>

<div class="outer">
    <div class="inner-left"></div>
    <div class="inner-center"></div>
    <div class="inner-right"></div>
</div>

```

方法二：圣杯布局

```

<style>
    .outer {
        position: absolute;
        top: 80px;
        right: 0;
        bottom: 80px;
        left: 0;
        padding: 0 250px;
    }

    .inner-left {
        float: left;
        width: 250px;
        margin-left: -250px;
    }

    .inner-right {
        float: right;
        width: 250px;
        margin-right: -250px;
    }
</style>

<!-- 注意下方结构的改变 -->
<div class="outer">
    <div class="inner-left"></div>
    <div class="inner-right"></div>
    <div class="inner-center"></div>
</div>

```

<!-- 思考：为什么在 html 部分 inner-right 会在 inner-center 前面呢 -->

方法三：双飞翼布局

```

<style>
    .outer {
        position: absolute;
        top: 80px;
        right: 0;
        bottom: 80px;
        left: 0;
    }

    .inner-left {
        float: left;
    }

```

```

        width: 250px;
    }

    .inner-right {
        float: right;
        width: 250px;
    }

    .inner-center {
        margin: 0 250px;
    }
}
</style>

<div class="outer">
    <div class="inner-left"></div>
    <div class="inner-right"></div>
    <div class="inner-center"></div>
</div>

```

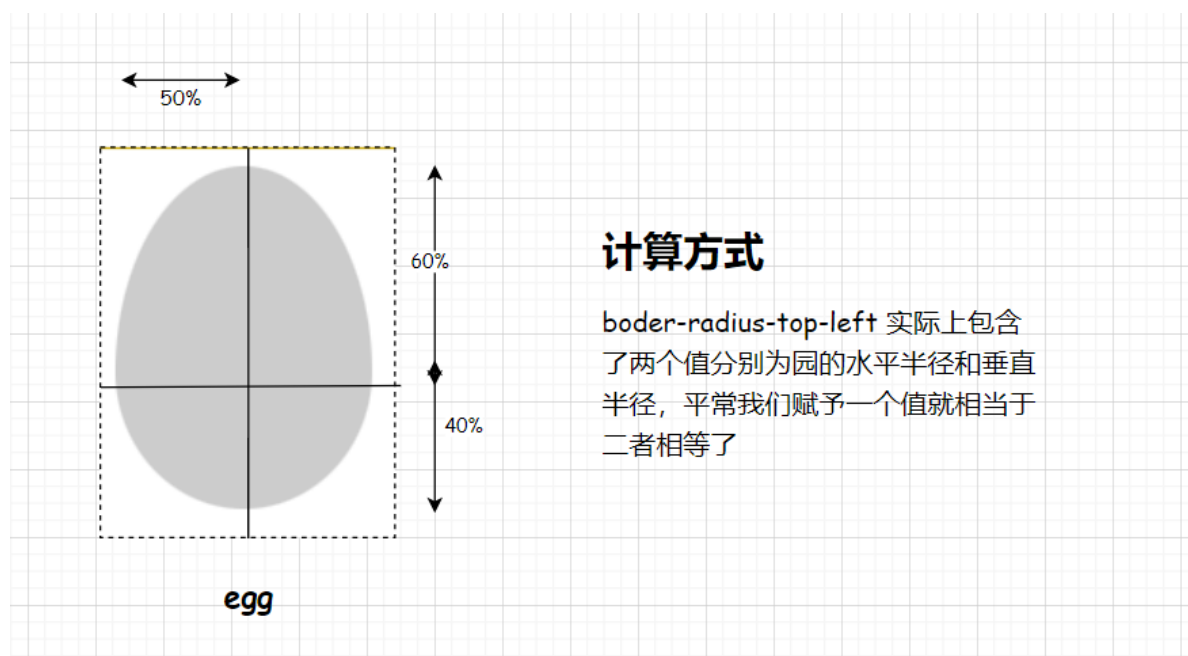
grid 方式也能轻松实现该布局，只需 outer 设置 `grid-template-column: 250px 1fr 250px` 便可实现，此处不再展开。

小结：在CSS3中为我们添加了弹性盒模型和栅格系统为我们的布局提供了很大的遍历，但这些经典的方式也有很多值得我们学习的地方，各种方式都掌握用起来才能游刃有余😊。

0.3 小案例

画出xxx图形，实现xxx效果

0.3.1 画一个鸡蛋



```

.egg {
    width: 120px;
    height: 160px;
    background-color: #ccc;
    border-radius: 50% 50% 50% 50% / 60% 60% 40% 40%;
}

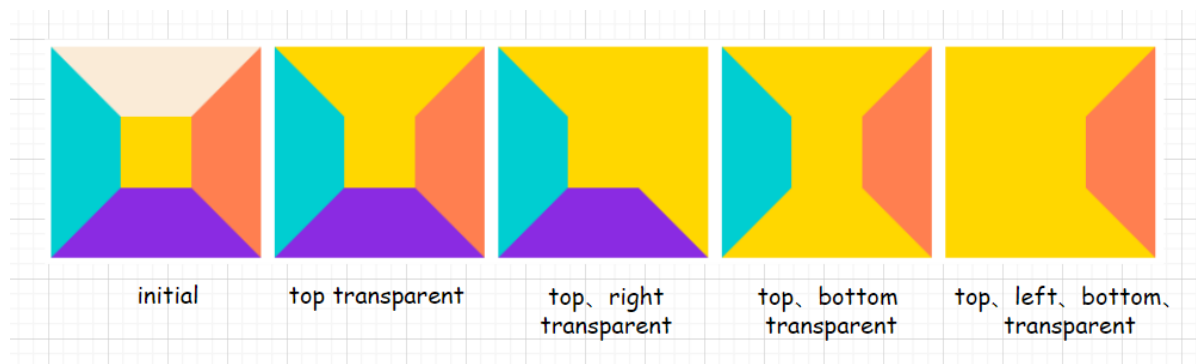
```

0.3.2 画一个三角形

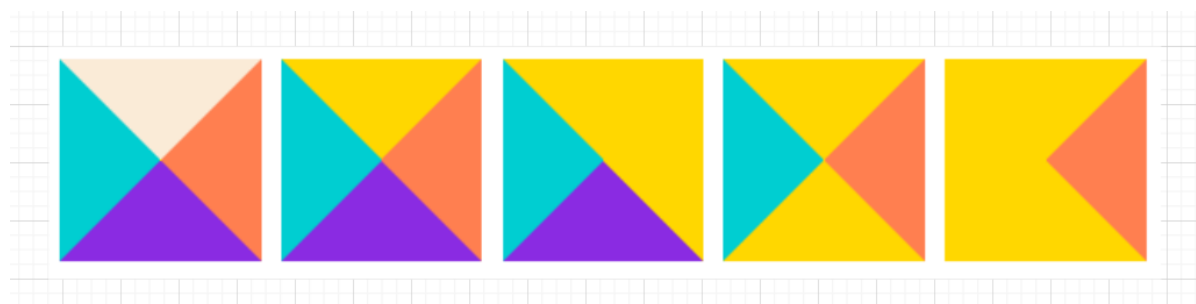
画三角形之前，我们先来看看 `border` 的一些表现，我们设置如下代码

```
.inner {  
  width: 50px;  
  height: 50px;  
  background-color: gold;  
  border-top: 50px solid antiquewhite;  
  border-left: 50px solid darkturquoise;  
  border-right: 50px solid coral;  
  border-bottom: 50px solid blueviolet;  
}
```

我们再分别隐藏一个到多个边。

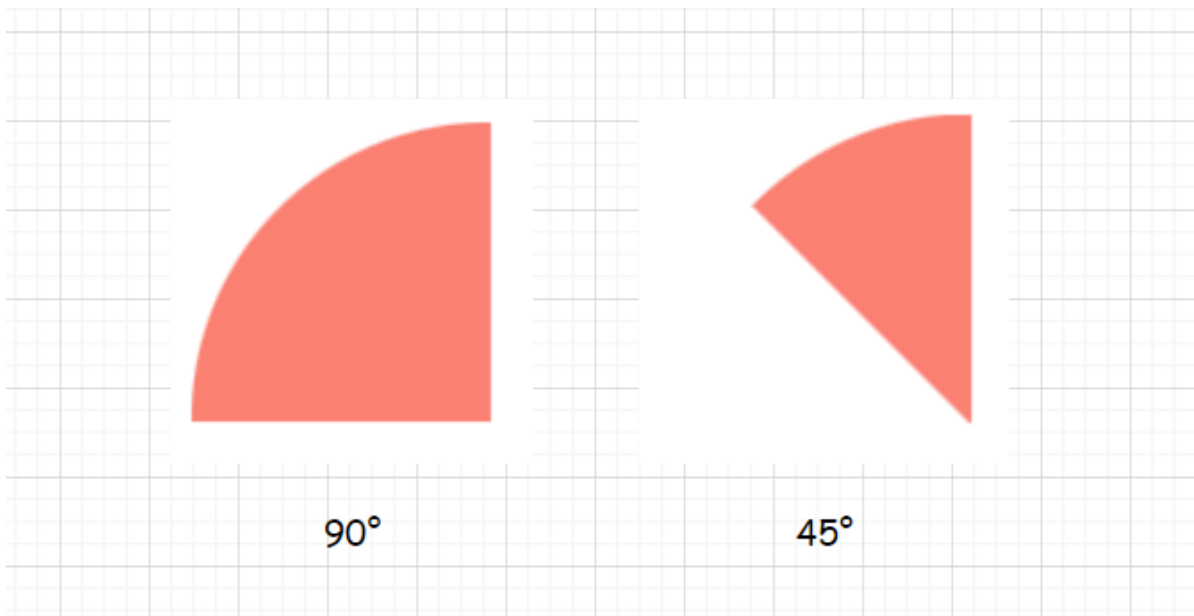


此时我们发现四个角都是由**两个三角形**拼接而成，此时我们把元素的宽高置零



当去掉黄色底色后，就得到了我们想要的图形，其中3 和 5 都可以作为结果，如果我们想要任意角度的画可以考虑 `transform` 中的 `scale()` 函数

0.3.3 画一个扇形



实现90°的扇形也可以用 `border-radius`，如果要实现任意角的话可以再考虑 `transform: rotate()` 代码如下：

```
.outer {
  overflow: hidden;
  width: 100px;
  height: 100px;
}

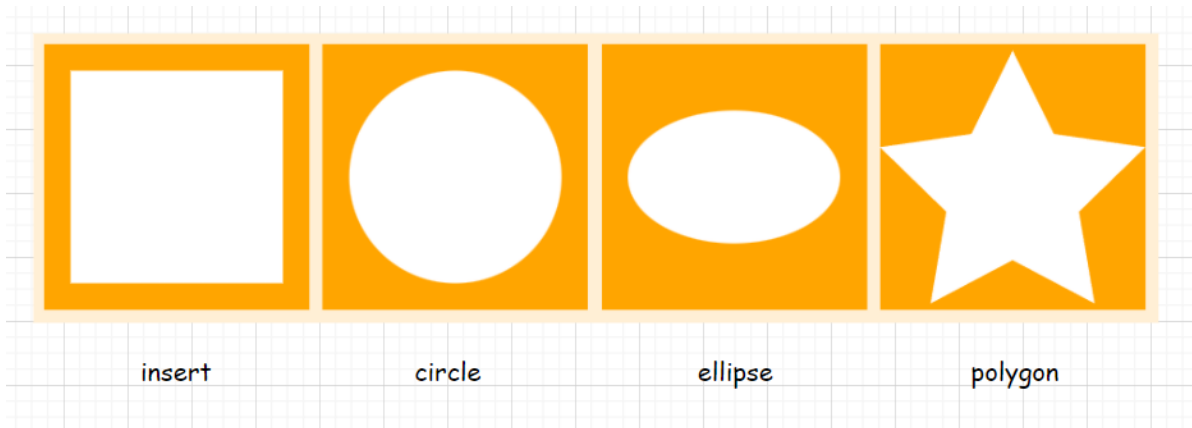
.inner {
  border-radius: 100px 0 0;
  width: 100%;
  height: 100%;
  background: salmon;
  transform-origin: right bottom;
  transform: rotate(45deg);
}
```

inner里面的代码也可以写成这样，和上方的实现三角形类似。

```
.inner {
  border: 100px solid transparent;
  width: 0;
  border-radius: 100px;
  border-left-color: skyblue;
}
```

0.3.4 画出任意图形

clip-path 使用裁剪方式创建元素的可显示区域。区域内的部分显示，区域外的隐藏。



```
.outer {
  margin-right: 10px;
  width: 200px;
  height: 200px;
  background-color: orange;
}

.inner {
  width: 200px;
  height: 200px;
  background-color: #fff;
}

/* 给inner分别加上 */
clip-path: inset(20px);
clip-path: circle(40%);
clip-path: ellipse(80px 50px);
clip-path: polygon(50% 2.4%, 34.5% 33.8%, 0% 38.8%, 25% 63.1%,
  19.1% 97.6%, 50% 81.3%, 80.9% 97.6%, 75% 63.1%, 100% 38.8%,
  65.5% 33.8%);
```

简单介绍一下这些函数的用法:

值	说明
<code>inset()</code>	创建出的矩形距离内容边界上、右、下、左的长度
<code>circle()</code>	接收两个参数半径和圆心，圆心缺省值为中心位置
<code>ellipse()</code>	接收三个参数分别为长轴和短轴的长度于圆心的位置
<code>polygon()</code>	多个参数代表顶点，每个参数都有两个值代表点的横纵坐标

可以通过这个网站生成图形来学习和使用: <https://bennettfeely.com/clippy/>

还有一个 `path()` 函数，可以实现类似SVG的语法这里不再展开，可以去MDN上看看 <https://developer.mozilla.org/zh-CN/docs/Web/SVG/Tutorial/Paths>

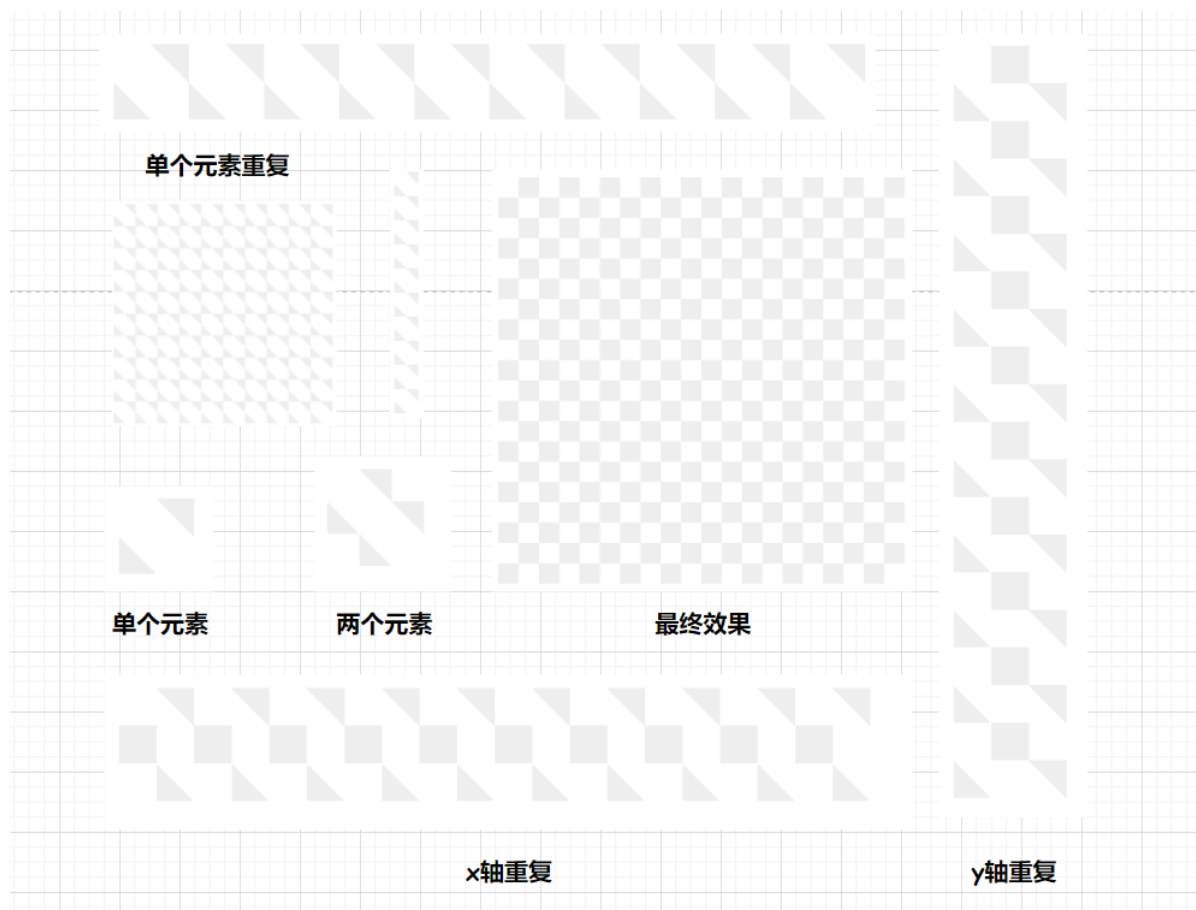
0.3.5 用CSS实现网格背景

通过背景渐变、重复等属性，就能实现如下的网格背景

```
.inner {
  width: 400px;
```

```
height: 400px;
background-image: linear-gradient(
  45deg,
  #eee 25%,
  transparent 25%,
  transparent 75%,
  #eee 75%
),
linear-gradient(
  45deg,
  #eee 25%,
  transparent 25%,
  transparent 75%,
  #eee 75%
);
background-position: 0 0, 20px 20px;
background-size: 40px 40px;
```

比划比划就能理解了



此处开始偏概念的知识点占大部分，但是最好还是手动都敲一敲。

1. 选择器

选择器	格式	优先级权重
id选择器	#id	100
类选择器	.outer	10
属性选择器	div[value="123"]	10
伪类选择器	a:hover	10
标签选择器	div	1
伪元素选择器	li::after	1
相邻兄弟选择器	h1+p	0
通用兄弟选择器	h1~p	0
子选择器	.outer > .inner	0
后代选择器	li a	0
通配符选择器	*	0

对于选择器的**优先级**：

- 标签选择器、伪元素选择器：1
- 类选择器、伪类选择器、属性选择器：10
- id 选择器：100
- 内联样式：1000

1.1 属性选择器语法

```
/* 存在title属性的<a> 元素 */
a[title] {
  color: purple;
}

/* 存在href属性并且属性值匹配"https://example.org"的<a> 元素 */
a[href="https://example.org"] {
  color: green;
}

/* 存在href属性并且属性值包含"example"的<a> 元素 */
a[href*="example"] {
  font-size: 2em;
}

/* 存在href属性并且属性值结尾是".org"的<a> 元素 */
a[href$=".org"] {
  font-style: italic;
}

/* 存在class属性并且属性值包含以空格分隔的"logo"的<a>元素 */
a[class~="logo"] {
  padding: 2px;
}
```

1.2 相邻与通用兄弟选择器

相邻兄弟选择器 (+) 介于两个选择器之间，当第二个元素紧跟在第一个元素之后，并且两个元素都是属于同一个父元素的子元素，则第二个元素将被选中。

通用兄弟选择器 (~) 位置无须紧邻，只须同层级，A~B 选择 A 元素之后所有同层级 B 元素。

注意事项：

- !important 声明的样式的优先级最高；
- 如果优先级相同，则最后出现的样式生效；
- 继承得到的样式的优先级最低；
- 通用选择器 (*)、子选择器 (>) 和相邻同胞选择器 (+) 并不在这四个等级中，所以它们的权值都为 0；
- 样式表的来源不同时，优先级顺序为：内联样式 > 内部样式 > 外部样式 > 浏览器用户自定义样式 > 浏览器默认样式。

2. 伪元素和伪类

2.1 伪元素

伪元素是一个附加至选择器末的关键词，允许你对被选择元素的特定部分修改样式。

```
selector::pseudo-element {
  property: value;
}
```

常用的伪元素：

伪元素	说明
::before	创建一个伪元素，其将成为匹配选中的元素的第一个子元素。常通过 content 属性来为一个元素添加修饰性的内容。此元素默认为行内元素。
::after	伪元素 ::after 用来创建一个伪元素，作为已选中元素的最后一个子元素。通常会配合 content 属性来为该元素添加装饰内容。这个虚拟元素默认是行内元素。
::first-letter	伪元素 ::first-letter 会选中某 block-level element（块级元素第一行的第一个字母，并且文字所处的行之前没有其他内容（如图片和内联的表格）。
::first-line	::first-line 伪元素在某块级元素的第一行应用样式。第一行的长度取决于很多因素，包括元素宽度，文档宽度和文本的文字大小。
::selection	::selection CSS 伪元素应用于文档中被用户高亮的部分（比如使用鼠标或其他选择设备选中的部分）。

注意：按照规范，应该使用双冒号 (::) 而不是单个冒号 (:)，以便区分伪类和伪元素。但是，由于旧版本的 W3C 规范并未对此进行特别区分，因此目前绝大多数的浏览器都同时支持使用这两种方式来表示伪元素。

2.2 伪类

CSS **伪类** 是添加到选择器的关键字，指定要选择的元素的特殊状态。例如，`:hover` 可被用于在用户将鼠标悬停在按钮上时改变按钮的颜色。

```
selector:pseudo-class {  
  property: value;  
}
```

常用伪类：

伪类	说明
<code>:active</code>	<code>:active</code> 匹配被用户激活的元素。它让页面能在浏览器监测到激活时给出反馈。当用鼠标交互时，它代表的是用户按下按键和松开按键之间的时间。
<code>:link</code>	<code>:link</code> 伪类选择器是用来选中元素当中的链接。它将会选中所有尚未访问的链接，包括那些已经给定了其他伪类选择器的链接
<code>:visited</code>	<code>:visited</code> 表示用户已访问过的链接。出于隐私原因，可以使用此选择器修改的样式非常有限。
<code>:hover</code>	<code>:hover</code> 适用于用户使用指示设备虚指一个元素（没有激活它）的情况。这个样式会被任何与链接相关的伪类重写
<code>:empty</code>	<code>:empty</code> 代表没有子元素的元素。子元素只可以是元素节点或文本（包括空格）。注释或处理指令都不会产生影响。
<code>:root</code>	<code>:root</code> 匹配文档树的根元素。对于 HTML 来说， <code>:root</code> 表示 html 元素，除了 优先级 更高之外，与 <code>html</code> 选择器相同。
<code>:not()</code>	<code>:not()</code> 用来匹配不符合一组选择器的元素。由于它的作用是防止特定的元素被选中，它也被称为 反选伪类 （ <i>negation pseudo-class</i> ）。
<code>:nth-child</code>	<code>:nth-child(an+b)</code> 首先找到所有当前元素的兄弟元素，然后按照位置先后顺序从1开始排序，选择的结果为CSS伪类:nth-child括号中表达式(an+b)匹配到的元素集合（n=0, 1, 2, 3...）。
<code>:nth-last-of-type</code>	<code>:nth-last-of-type(an+b)</code> 匹配那些在它之后有 $a*n+b-1$ 个相同类型兄弟节点的元素，其中 n 为正值或零值。它基本上和 <code>:nth-of-type</code> 一样，只是它从 结尾 处反序计数，而不是从开头处。
<code>:only-child</code>	<code>:only-child</code> 匹配没有任何兄弟元素的元素.等效的选择器还可以写成 <code>:first-child:last-child</code> 或者 <code>:nth-child(1):nth-last-child(1)</code> ,当然,前者的权重会低一点.
<code>:only-of-type</code>	<code>:only-of-type</code> 代表了任意一个元素，这个元素没有其他相同类型的兄弟元素。
表单伪类	<code>:valid</code> <code>:required</code> <code>:checked</code> <code>:disabled</code> <code>:focus</code> 等

注意：为保证样式生效，需要把 `:active` 的样式放在所有链接相关的样式之后。这种链接伪类先后顺序被称为 **LVHA** 顺序：`:link` — `:visited` — `:hover` — `:active`。

3. 盒模型

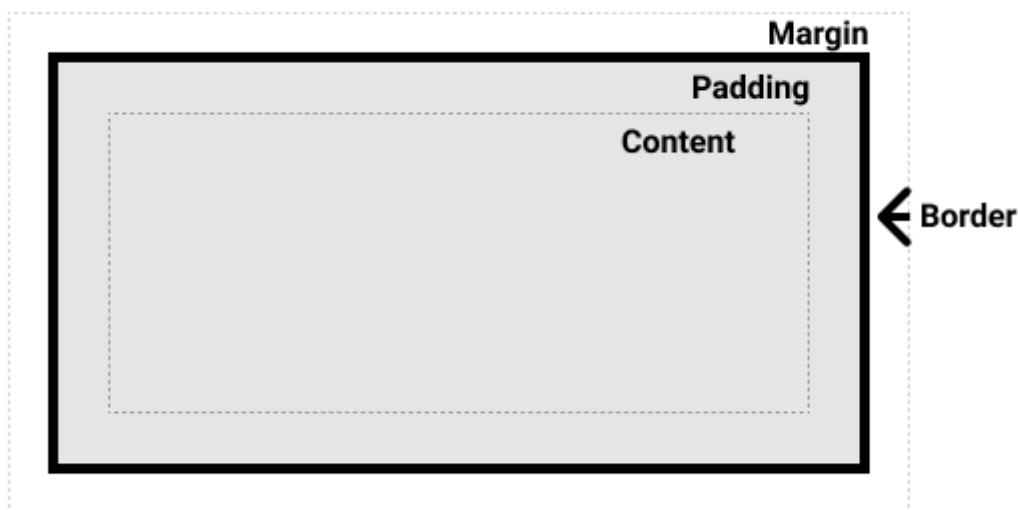
在 CSS 中，所有的元素都被一个个的“盒子 (box)”包围着，理解这些“盒子”的基本原理，是我们使用 CSS 实现准确布局、处理元素排列的关键。

3.1 块级盒子和内联盒子

在 CSS 中我们广泛地使用两种“盒子”——**块级盒子 (block box)** 和 **内联盒子 (inline box)**。这两种盒子会在**页面流 (page flow)** 和**元素之间的关系**方面表现出不同的行为

一个被定义成块级的 (block) 盒子会表现出以下行为:

- 盒子会在内联的方向上扩展并占据父容器在该方向上的所有可用空间，在绝大多数情况下意味着盒子会和父容器一样宽
- 每个盒子都会换行
- width 和 height 属性将不起作用。
- 内边距 (padding), 外边距 (margin) 和 边框 (border) 会将其他元素从当前盒子周围“推开”



除非特殊指定，诸如标题 (`<h1>` 等) 和段落 (`<p>`) 默认情况下都是块级的盒子。

如果一个盒子对外显示为 `inline`，那么他的行为如下：

- 盒子不会产生换行。
- width 和 height 属性将不起作用。
- 垂直方向的内边距、外边距以及边框会被应用但是不会把其他处于 `inline` 状态的盒子推开。
- 水平方向的内边距、外边距以及边框会被应用且会把其他处于 `inline` 状态的盒子推开。

用做链接的 `<a>` 元素、`<span>`、`<em>` 以及 `<strong>` 都是默认处于 `inline` 状态的。

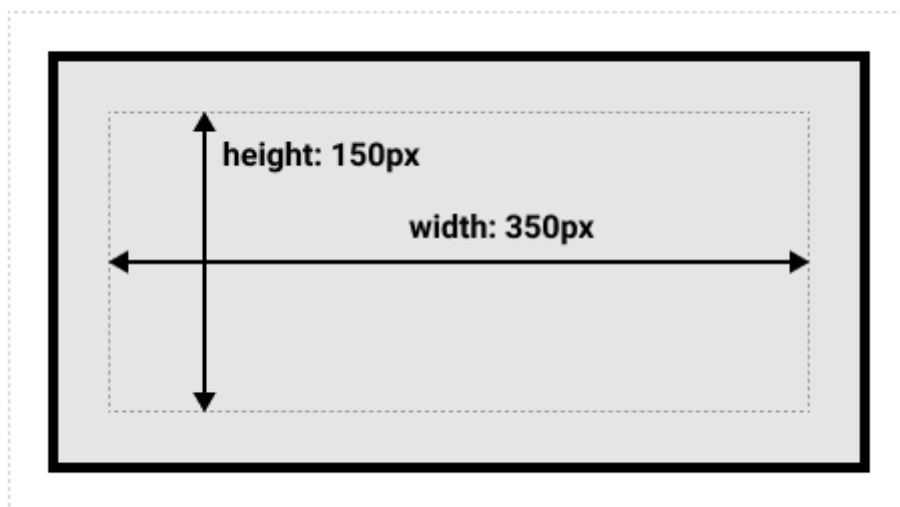
我们通过对盒子 `display` 属性的设置，比如 `inline` 或者 `block`，来控制盒子的外部显示类型。

3.2 CSS盒模型

完整的 CSS 盒模型应用于块级盒子，内联盒子只使用盒模型中定义的部分内容。模型定义了盒的每个部分——margin, border, padding, and content——合在一起就可以创建我们在页面上看到的内容。为了增加一些额外的复杂性，有一个**标准的**和**替代 (IE) 的**盒模型。

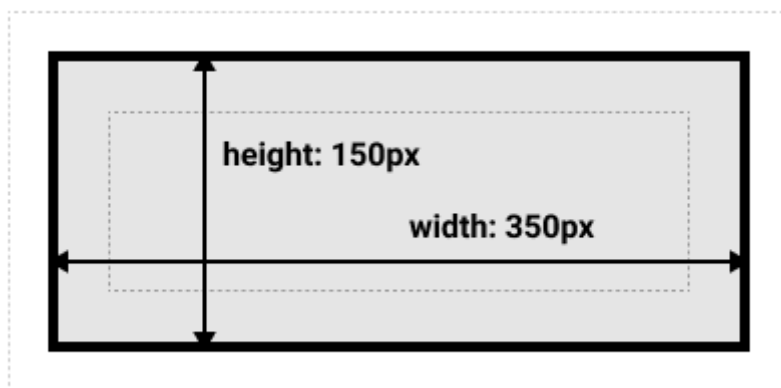
标准盒模型

在标准模型中，如果你给盒设置 `width` 和 `height`，实际设置的是 *content box*。padding 和 border 再加上设置的宽高一起决定整个盒子的大小。见下图。



替代 (IE) 盒模型

CSS还有一个替代盒模型。使用这个模型，所有宽度都是可见宽度，所以内容宽度是该宽度减去边框和填充部分。如果需要使用替代模型，可以通过为其设置 `box-sizing: border-box` 来实现。这样就可以告诉浏览器使用 `border-box` 来定义区域，从而设定您想要的大小。



3.3 外边距折叠

块的上外边距(`margin-top`)和下外边距(`margin-bottom`)有时合并(折叠)为单个边距，其大小为单个边距的最大值(或如果它们相等，则仅为其中一个)，这种行为称为**边距折叠**。

注意有设定 `float` 和 `position=absolute` 的元素不会产生外边距重叠行为。

3.3.1 同一层相邻元素之间

相邻的两个元素之间的外边距重叠

```
<style>
p:nth-child(1){
  margin-bottom: 13px;
}
p:nth-child(2){
  margin-top: 87px;
}
</style>

<p>下边界范围会...</p>
<p>...会跟这个元素的上边界范围重叠。</p>
```

这个例子如果以为边界会合并的话，理所当然会猜测上下2个元素会合并一个100px的边界范围，但实际会发生边界折叠，**只会挑选最大边界范围留下**，所以这个例子的边界范围其实是87px。

3.3.2 没有将父元素和后代元素分开

如果没有边框，边距，行内内容，也没有创建块级格式上下文或清除浮动来分开一个块级元素的上边界 `margin-top` 与其内一个或多个后代块级元素的上边界 `margin-top`；

或没有边框，内边距，行内内容，高度，来分开一个块级元素的下边界 `margin-bottom`；

则就会出现父块元素和其内后代块元素外边界重叠，重叠部分最终会溢出到父级块元素外面。

3.3.3 空的块元素

当一个块元素上边界 `margin-top` 直接贴到元素下边界 `margin-bottom` 时也会发生边界折叠。这种情况会发生在块元素完全没有设定边框 `border`、内边距 `padding`、高度 `height`、最小高度 `min-height`、最大高度 `max-height`、内容设定为 `inline` 或是加上 `clear-fix` 的时候。

一些需要注意的地方：

- 上述情况的组合会产生更复杂的外边距折叠。
- 即使某一外边距为0，这些规则仍然适用。因此就算父元素的外边距是0，第一个或最后一个子元素的外边距仍然会“溢出”到父元素的外面。
- 如果参与折叠的外边距中包含负值，折叠后的外边距的值为最大的正边距与最小的负边距（即绝对值最大的负边距）的和；也就是说如果有 -13px 8px 100px 叠在一起，边界范围的就是 100px - 13px 的 87px。
- 如果所有参与折叠的外边距都为负，折叠后的外边距的值为最小的负边距的值。这一规则适用于相邻元素和嵌套元素。

以上这些内容都是发生在Block-Level的元素，设定floating和absolutely positioned的元素完全不用担心边界重叠的问题。

flex与grid等盒子在后续中说明

4. 浮动与清除浮动

`float` 属性最初只用于在成块的文本内浮动图像，但是现在它已成为在网页上创建多列布局的最常用工具之一。

4.1 float

`float` CSS属性指定一个元素应沿其容器的左侧或右侧放置，允许文本和内联元素环绕它。该元素从网页的正常流动(文档流)中移除，尽管仍然保持部分的流动性。

语法：

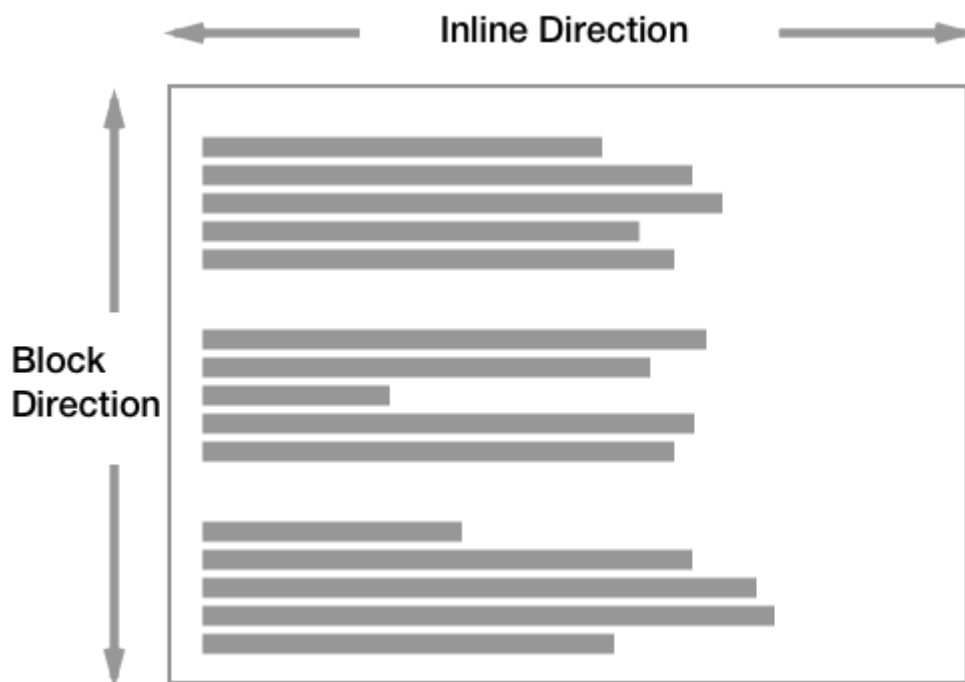
值	说明
<code>left</code> <code>right</code>	表明元素浮动在块容器的左 右侧
<code>inline-start</code> <code>end</code>	表明元素必须浮动在其所在块容器的开始 结束一侧
<code>none</code>	不进行浮动

4.1.1 浮动元素是如何定位的？

正如我们前面提到的那样，当一个元素浮动之后，它会被移出正常的文档流，然后向左或者向右平移，一直平移直到碰到了所处的容器的边框，或者碰到**另外一个浮动的元素**。

4.1.2 文档流

- 在正常的文档流中，盒模型会按块元素从上到下依次排列，行内元素会从左到右依次排列
- 在 html 文件输入文本，多个空格会被合并成一个空格显示到浏览器页面中。这种现象称为**空白折叠现象**。
- 文字还有图片大小不一，都会让我们页面的元素出现高矮不齐的现象，但是在浏览器查看我们的页面总会发现底边对齐；
- 每个块级元素会在上一个元素下面**另起一行**，它们会被设置好的margin 分隔。在英语，或者其他水平书写、自上而下模式里，块级元素是垂直组织的。
- 内联元素的表现有所不同 --- 它们不会另起一行；只要在其父级块级元素的宽度内有足够的空间，它们与其他内联元素、相邻的文本内容（或者被包裹的）被安排在同一行。如果空间不够，溢出的文本或元素将移到**新的一行**。



4.1.3 常见应用

首字下沉，文字环绕，两列布局，三列布局，圣杯布局等等。

4.2 清除浮动

为什么要清除浮动：浮动的元素会脱离文档流，当一个元素脱离文档流后，其在父元素所占的空间会坍塌导致父元素的高度塌陷，其后方的兄弟元素位置会前移，有时候我们不希望出现这样的现象。

4.2.1 父元素脱离文档流

让父元素也浮动起来或者添加 `position: absolute` 等

此时父元素也脱离了文档流，在有些时候我们能用上，但是这种方法会让**父元素的父元素**也出现高度坍塌，这可能不是我们想要的。

4.2.2 clear

用法：

值	说明
<code>none</code>	元素不会向下移动清除之前的浮动。
<code>left</code> <code>right</code>	元素被向下移动用于清除之前的左 右浮动。
<code>inline-start</code> <code>end</code>	该关键字表示该元素向下移动以清除其包含块的起始侧上的浮动。即在某个区域的左侧浮动或右侧浮动。
<code>both</code>	元素被向下移动用于清除之前的左右浮动。

当应用于**非浮动块**时，它将非浮动块的边框边界移动到所有相关浮动元素外边界的下方。这个非浮动块的垂直外边距会折叠。

下面给出清除浮动的应用

空元素：在需要清除浮动位置的后方添加一个空元素，给其添加相应的样式

```
.clearfix {
  display: block;
  height: 0;
  clear: both;
}
```

此方法需要手动创建一个空元素，我们可以通过 `::after` 直接让父元素在最后添加清除浮动的伪元素

```
.clearfix::after {
  content: '';
  display: block;
  height: 0;
  clear: both;
}
```

4.2.3 父元素定宽高

这个就比较明显了，此处不再展开

4.2.4 触发BFC机制

下一节展开说明

4.3 BFC 块格式化上下文

块格式化上下文 (Block Formatting Context, BFC) 是Web页面的可视CSS渲染的一部分，是块盒子的布局过程发生的区域，也是浮动元素与其他元素交互的区域。

它决定了元素如何对其内容进行定位，以及与其它元素的关系和相互作用，当涉及到可视化布局时，提供了一个环境，HTML 在这个环境中按照一定的规则进行布局。

浮动定位和清除浮动时只会应用于同一个BFC内的元素。浮动不会影响其它BFC中元素的布局，而清除浮动只能清除同一BFC中在它前面的元素的浮动。**外边距折叠 (Margin collapsing)** 也只会发生在属于同一BFC的块级元素之间。

下列方式会创建块格式化上下文：

- 根元素 (html)
- 浮动元素 (元素的 float 不是 none)
- 绝对定位元素 (元素的 position 为 absolute 或 fixed)
- 行内块元素 (元素的 display 为 inline-block)
- 表格单元格 (元素的 display 为 table-cell, HTML表格单元格默认为该值)
- 表格标题 (元素的 display 为 table-caption, HTML表格标题默认为该值)
- 匿名表格单元格元素 (元素的 display 为 table、table-row、table-row-group、table-header-group、table-footer-group (分别是HTML table、row、tbody、thead、tfoot 的默认属性) 或 inline-table)
- overflow 计算值(Computed)不为 visible 的块元素
- display 值为 flow-root 的元素
- contain 值为 layout、content 或 paint 的元素
- 弹性元素 (display 为 flex 或 inline-flex 元素的直接子元素)
- 网格元素 (display 为 grid 或 inline-grid 元素的直接子元素)

使用 overflow: auto

创建一个会包含这个浮动的 BFC，通常的做法是设置父元素 `overflow: auto` 或者设置其他的非默认的 `overflow: visible` 的值。

设置 `overflow: auto` 创建一个新的BFC来包含这个浮动。我们的 `<div>` 元素现在变成布局中的迷你布局。任何子元素都会被包含进去。

使用 `overflow` 来创建一个新的 BFC，是因为 `overflow` 属性告诉浏览器你想要怎样处理溢出的内容。当你使用这个属性只是为了创建 BFC 的时候，你可能会发现一些不想要的问题，比如滚动条或者一些剪切的阴影，需要注意。另外，对于后续的开发，可能不是很清楚当时为什么使用 `overflow`。所以你最好添加一些注释来解释为什么这样做。

使用 display: flow-root

一个新的 `display` 属性的值，它可以创建无副作用的 BFC。在父级块中使用 `display: flow-root` 可以创建新的 BFC。

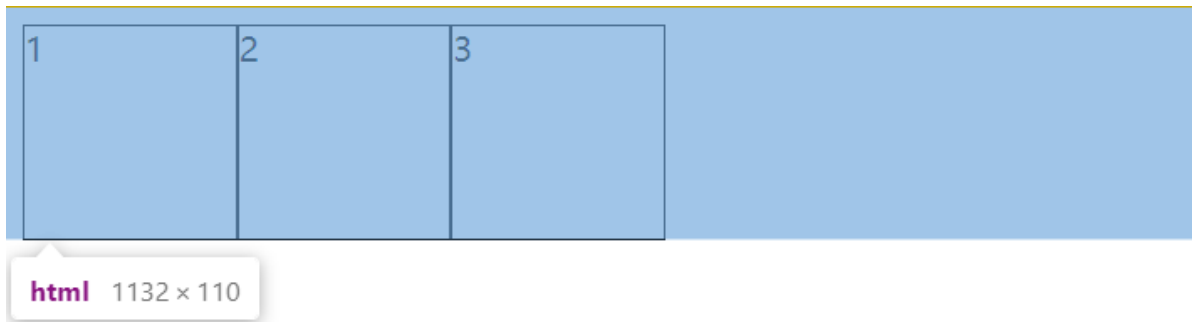
给 `<div>` `display: flow-root` 属性后，`<div>` 中的所有内容都会参与 BFC，浮动的内容不会从底部溢出。

关于值 `flow-root` 的这个名字，当你明白你实际上是在创建一个行为类似于根元素（浏览器中的 `<html>` 元素）的东西时，就能发现这个名字的意义了——即创建一个上下文，里面将进行 flow layout。

BFC的行为与**最外层的文档**非常相似，它在主布局中创造了一个小布局。BFC包含其内部的所有内容，`float` 和 `clear` 仅适用于**同一格式上下文**中的项目，而页边距仅在**同一格式上下文**中的元素之间折叠。

总结：我们在理解BFC时不需要硬背他的概念，其实正如上文所说的，BFC的行为与最外层文档 `html` 是非常相似的，如下所示

```
section {  
  background-color: blueviolet;  
}  
  
div {  
  width: 100px;  
  height: 100px;  
  background-color: #fff;  
  border: 1px solid #000;  
  float: left;  
}
```



正如前面所说，`html` 标签本身就符合BFC的特征，我们没有给它高度或者清除浮动，但此时浮动的元素是包含在这里面的。

5. 布局和包含块

一个元素的尺寸和位置经常受其**包含块(containing block)**的影响。大多数情况下，包含块就是这个元素最近的祖先块元素的内容区，但也不是总是这样

元素的尺寸及位置，常常会受它的包含块所影响。对于一些属性，例如 `width`, `height`, `padding`, `margin`，绝对定位元素的偏移值（比如 `position` 被设置为 `absolute` 或 `fixed`），当我们对其赋予**百分比**值时，这些值的计算值，就是通过元素的包含块计算得来。

5.1 包含块的确定

确定一个元素的包含块的过程完全依赖于这个元素的 `position` 属性：

- 如果 `position` 属性为 **static**、**relative** 或 **sticky**，包含块可能由它的最近的祖先块元素（比如说 `inline-block`, `block` 或 `list-item` 元素）的内容区的边缘组成，也可能会建立格式化上下文(比如说 `a table container`, `flex container`, `grid container`, 或者是 `the block container` 自身)。
- 如果 `position` 属性为 **absolute**，包含块就是由它的最近的 `position` 的值不是 **static**（也就是值为 `fixed`, `absolute`, `relative` 或 `sticky`）的祖先元素的内边距区的边缘组成。
- 如果 `position` 属性是 **fixed**，在连续媒体的情况下(`continuous media`)包含块是 **viewport**，在分页媒体(`paged media`)下的情况下包含块是**分页区域(page area)**。
- 如果 `position` 属性是 **absolute** 或 **fixed**，包含块也可能是由满足以下条件的最近父级元素的内边距区的边缘组成的：

- **transform** 或 **perspective** 的值不是 **none**
 - will-change 的值是 transform 或 perspective
 - filter 的值不是 none 或 will-change 的值是 filter(只在 Firefox 下生效).
 - contain 的值是 paint (例如: contain: paint;)

5.2 根据包含块计算百分值

如上所述，如果某些属性被赋予一个百分值的话，它的计算值是由这个元素的包含块计算而来的。

这些属性包括盒模型属性和偏移属性：

1. 要计算 **height**, **top**, **bottom** 中的百分值，是通过包含块的 **height** 的值。如果包含块的 **height** 值会根据它的内容变化，而且包含块的 **position** 属性的值被赋予 **relative** 或 **static**，那么，这些值的计算值为 **auto**。
2. 要计算 **width**, **left**, **right**, **padding**, **margin** 这些属性由包含块的 **width** 属性的值来计算它的百分值。

6. 显示和隐藏

display 属性可以设置元素的内部和外部显示类型 *display types*。元素的外部显示类型 *outer display types* 将决定该元素在[流式布局](#)中的表现（块级或内联元素）；元素的内部显示类型 *inner display types* 可以控制其子元素的布局

在 **CSS 规范**（特指 CSS Level 1/2/3 规范）中查阅 **display** 属性的所有取值时需要注意：个别取值的详细信息记录在独立的规范中。

6.1 显示&display

<display-outside> 指定元素的外部显示类型: **block**, **inline**

<display-inside> 指定元素的内部显示类型，它们定义了该元素内部内容的布局方式:

值	说明
flow-root	生成一个块元素，创建块级格式上下文，定于格式化所在的根位置。
table	生成一个块元素，行为类似html中的table。
flex	生成一个块元素，并对内容采用弹性盒子模型进行布局。
grid	生成一个块元素，并对内容采用网格模型进行布局。

<display-listitem> 将这个元素的外部显示类型变为 **block** 盒，并将内部显示类型变为多个 **list-item inline** 盒。

<display-legacy> CSS 2使用单个关键字来指定display的属性，对于相同布局模式的 **block** 级和 **inline** 级变体需要使用单独的关键字。

值	说明
<code>inline-block</code>	元素会产生一个块元素盒子，并且像内联盒子一样（表现得更像一个被替换的元素），可以融入到周围内容中。等同于 <code>inline flow-root</code> 。
<code>inline-table</code>	在HTML中， <code>inline-table</code> 没有直接对应关系。它表现为一个元素，但是又表现为一个不同于块级盒子的内联盒子。表盒子内部是一个块级上下文。等同于 <code>inline table</code> 。
<code>inline-flex</code>	元素表现为一个内联元素，并对内容采用弹性盒子模型进行布局。等同于 <code>inline flex</code> 。
<code>inline-grid</code>	元素表现为一个内联元素，并对内容采用网格模型进行布局。等同于 <code>inline grid</code> 。

注意：在一个`display`中定义两个关键字来指定`display-outside`和`display-inside`的语法在CSS3中可使用，但是很多浏览器还不支持该写法，有兼容性问题。可以用`display-legacy`来指定想要的效果。

6.2 隐藏

此处介绍几种将元素隐藏的方式

6.2.1 `display: none;`

将 `display` 设置为 `none` 会将元素从 [可访问性树 accessibility tree](#) 中移除。这会导致该元素及其所有子代元素不再被屏幕阅读技术 *screen reading technology* 访问。

如果你只是想从视觉上隐藏这个元素，对可访问性更加友好的做法是使用 属性组合 来将其从屏幕上隐藏，但仍可以被屏幕阅读器 *screen readers* 等辅助技术解析。

6.2.2 `visiblity`

CSS属性 `visibility` 显示或隐藏元素而不更改文档的布局。该属性还可以隐藏 `<table>` 中的行或列。

值	说明
<code>visible</code>	元素正常显示。
<code>hidden</code>	隐藏元素，但是其他元素的布局不改变，相当于此元素变成透明。要注意若将其子元素设为 <code>visibility: visible</code> ，则该子元素依然可见。

6.3 溢出的内容

盒子内的空间不够大时内部的元素发生溢出，我们需要对溢出内容的去留或是显示的方式做出行动，控制在我们期待的效果之中。

只要有可能，CSS就不会隐藏你的内容，隐藏引起的数据损失通常会造成困扰。在CSS的术语里面，这会导致一些内容消失，你的访客可能不会注意到这一点，如果消失的是表格上的提交按钮，没有人能填完这个表格，这是很麻烦的事情！所以CSS反而会把它以可见的形式溢出出去。这样做的结果就是，你会看到错误的CSS导致的一片混乱，或者最坏的情况也只是你的网站的访客会告诉你有些内容冒了出来，你的网站需要修缮。

6.3.1 overflow 属性

overflow属性是你控制一个元素溢出的方式，它告诉浏览器你想怎样处理溢出。overflow的默认值为visible，这就是我们的内容溢出的时候，我们在默认情况下看到它们的原因。

值	说明
visible	默认值。内容不会被修剪，可以呈现在元素框之外。
hidden	如果需要，内容将被剪裁以适合填充框。不提供滚动条。
scroll	如果需要，内容将被剪裁以适合填充框。浏览器显示滚动条，无论是否实际剪切了任何内容。（这可以防止滚动条在内容更改时出现或消失。）打印机仍可能打印溢出的内容。
auto	取决于用户代理。如果内容适合填充框内部，则它看起来与可见内容相同，但仍会建立新的块格式化上下文。如果内容溢出，桌面浏览器会提供滚动条。

通常情况下我们可能只会在一个方向上滚动或隐藏，此时我们可以选择overflow-x 或者overflow-y 否则overflow-x与overflow-y都会被设置为相同的值

滚动条 scroll样式 的控制

该特性是非标准的，请尽量不要在生产环境中使用它！

你可以使用以下伪元素选择器去修改各式 webkit 浏览器的滚动条样式:

- `::-webkit-scrollbar` — 整个滚动条.
- `::-webkit-scrollbar-button` — 滚动条上的按钮 (上下箭头).
- `::-webkit-scrollbar-thumb` — 滚动条上的滚动滑块.
- `::-webkit-scrollbar-track` — 滚动条轨道.
- `::-webkit-scrollbar-track-piece` — 滚动条没有滑块的轨道部分.
- `::-webkit-scrollbar-corner` — 当同时有垂直滚动条和水平滚动条时交汇的部分.
- `::-webkit-resizer` — 某些元素的corner部分的部分样式(例:textarea的可拖动按钮).

6.3.2 overflow-wrap

CSS 属性 overflow-wrap 是用来说明当一个不能被分开的字符串太长而不能填充其包裹盒时，为防止其溢出，浏览器是否允许这样的单词中断换行。与word-break相比，overflow-wrap仅在无法将整个单词放在自己的行而不会溢出的情况下才会产生中断。word-wrap 现在被当作overflow-wrap 的“别名”。

overflow-wrap

值	说明
normal	行只能在正常的单词断点处中断。
break-word	表示如果行内没有多余的地方容纳该单词到结尾，则那些正常的不能被分割的单词会被强制分割换行。

word-break

值	说明
<code>normal</code>	使用默认的断行规则。
<code>break-all</code>	对于non-CJK (CJK 指中文/日文/韩文) 文本，可在任意字符间断行。
<code>keep-all</code>	CJK 文本不断行。 Non-CJK 文本表现同 <code>normal</code> 。

6.3.3 white-space

`white-space` CSS 属性是用来设置如何处理元素中的 空白

常用值：

值	说明
<code>normal</code>	连续的空白符会被合并，换行符会被当作空白符来处理。
<code>nowrap</code>	和 <code>normal</code> 一样，连续的空白符会被合并。但文本内的换行无效。
<code>pre</code>	连续的空白符会被保留。在遇到换行符或者 <code>
</code> 元素时才会换行。

下面的表格总结了各种 `white-space` 值的行为：

值	换行符	空格和制表符	文字换行	行尾空格
<code>normal</code>	合并	合并	换行	删除
<code>nowrap</code>	合并	合并	不换行	删除
<code>pre</code>	保留	保留	不换行	保留
<code>pre-wrap</code>	保留	保留	换行	挂起
<code>pre-line</code>	保留	合并	换行	删除
<code>break-spaces</code>	保留	保留	换行	换行

6.3.4 text-overflow

`text-overflow` CSS 属性确定如何向用户发出未显示的溢出内容信号。它可以被剪切，显示一个省略号 ('...', U + 2026 HORIZONTAL ELLIPSIS) 或显示一个自定义字符串。

这个属性只对那些在块级元素溢出的内容有效，但是必须要与块级元素内联(inline)方向一致（举个反例：内容在盒子的下方溢出。此时就不会生效）。文本可能在以下情况下溢出：当其因为某种原因而无法换行(例子：设置了 `white-space:nowrap`)，或者一个单词因为太长而不能合理地被安置(fit)。

这个属性并不会强制“溢出”事件的发生，因此为了能让 `text-overflow` 能够生效，程序员们必须要在元素上添加几个额外的属性，比如“将 `overflow` 设置为 `hidden`”。

Visible overflow

Clipped overflow

Ellipsis

Custom string "."

Custom string ""

值	说明
<code>clip</code>	此为默认值。 这个关键字的意思是“在内容区域的极限处截断文本”，因此在字符的中间可能会发生截断。如果你的目标浏览器支持 <code>text-overflow: ''</code> ，为了能在两个字符过渡处截断，你可以使用一个空字符串值 ('') 作为 <code>text-overflow</code> 属性的值。
<code>ellipsis</code>	这个关键字的意思是“用一个省略号 ('...', U+2026 HORIZONTAL ELLIPSIS)来表示被截断的文本”。这个省略号被添加在内容区域中，因此会减少显示的文本。如果空间太小到连省略号都容纳不下，那么这个省略号也会被截断。

[其他值](#)可查看MDN说明

7. 值与单位

CSS值倾向于使用尖括号表示，以区别于CSS属性(例如 `color` 属性和 `<color>` 数据类型)。

7.1 数字、长度和百分比

以下全部归类为数值：

数值类型	描述
<code><integer></code>	<code><integer></code> 是一个整数，比如1024或-55。
<code><number></code>	<code><number></code> 表示一个小数——它可能有小数点后面的部分，也可能没有，例如0.255、128或-1.2。
<code><dimension></code>	<code><dimension></code> 是一个 <code><number></code> ，它有一个附加的单位，例如45deg、5s或10px。 <code><dimension></code> 是一个伞形类别，包括 <code><length></code> 、 <code><angle></code> 、 <code><time></code> 和 <code><resolution></code> 类型。
<code><percentage></code>	<code><percentage></code> 表示一些其他值的一部分，例如50%。百分比值总是相对于另一个量，例如，一个元素的长度相对于其父元素的长度。

7.1.1 长度

最常见的数字类型是 `<length>`，例如10px(像素)或30em。CSS中有两种类型的长度——相对长度和绝对长度。重要的是要知道它们之间的区别，以便理解他们控制的元素将变得有多大。

绝对长度单位

以下都是**绝对**长度单位——它们与其他任何东西都没有关系，通常被认为总是相同的大小。

cm	厘米	1cm = 96px/2.54
mm	毫米	1mm = 1/10th of 1cm
Q	四分之一毫米	1Q = 1/40th of 1cm
in	英寸	1in = 2.54cm = 96px
pc	十二点活字	1pc = 1/16th of 1in
pt	点	1pt = 1/72th of 1in
px	像素	1px = 1/96th of 1in

相对长度单位

相对长度单位相对于其他一些东西，比如父元素的字体大小，或者视图端口的大小。使用相对单位的好处是，经过一些仔细的规划，您可以使文本或其他元素的大小与页面上的其他内容相对应。下表列出了web开发中一些最有用的单位。

单位	相对于
em	在 font-size 中使用是相对于父元素的字体大小，在其他属性中使用是相对于自身的字体大小，如 width
ex	字符“x”的高度
ch	数字“0”的宽度
rem	根元素的字体大小
lh	元素的line-height
vw	视窗宽度的1%
vh	视窗高度的1%
vmin	视窗较小尺寸的1%
vmax	视图大尺寸的1%

em 和 rem 在排版属性中

- em 单位的意思是“父元素的字体大小”
- rem 单位的意思是“根元素的字体大小”

如果在CSS中更改 `<html>` 字体大小，将看到所有其他相关内容都发生了更改，包括 rem 和 em 大小的文本。

7.1.2 百分比

在许多情况下，百分比与长度的处理方法是一样的。百分比的问题在于，它们总是相对于其他值设置的。例如，如果将元素的字体大小设置为百分比，那么它将是元素父元素字体大小的百分比。如果使用百分比作为宽度值，那么它将是父值宽度的百分比。

7.1.3 数字

有些值接受数字，不添加任何单位。接受无单位数字的属性的一个例子是不透明度属性（`opacity`），它控制元素的不透明度(它的透明程度)。此属性接受0(完全透明)和1(完全不透明)之间的数字。

7.2 颜色

在CSS中指定颜色的方法有很多，其中一些是最近才实现的。在CSS中，相同的颜色值可以在任何地方使用，无论指定的是文本颜色、背景颜色还是其他颜色。

7.2.1 颜色关键词

简易地指定了各种颜色的关键字，详情可查阅 https://developer.mozilla.org/en-US/docs/Web/CSS/color_value

7.2.2 十六进制RGB与RGBA

每个十六进制值由一个散列/磅符号(#)和六个十六进制数字组成，每个十六进制数字都可以取0到f(代表15)之间的16个值中的一个——所以是0123456789abcdef。每对值表示一个通道——红色、绿色和蓝色——并允许我们为每个通道指定256个可用值中的任意一个(16 x 16 = 256)。

RGBA的工作方式与RGB颜色完全相同，因此可以使用任何RGB值，但是有第四个值表示颜色的alpha通道，它控制不透明度。如果将这个值设置为0，它将使颜色完全透明，而设置为1将使颜色完全不透明。介于两者之间的值提供了不同级别的透明度。

7.3 图片

`<image>` 数据类型用于图像为有效值的任何地方。它可以是一个通过 `url()`函数指向的实际图像文件，也可以是一个渐变。

常见的可用属性

属性	说明
<code>background-image</code>	背景图片
<code>list-style-image</code>	列表前缀样式
<code>border-image-source</code>	边框的背景图样式
<code>cursor</code>	鼠标的图片样式

7.4 位置

`<position>` 数据类型表示一组2D坐标，用于定位一个元素，如背景图像(通过 `background-position`)。它可以使用关键字(如 `top`, `left`, `bottom`, `right`, 以及 `center`)将元素与2D框的特定边界对齐，以及表示框的顶部和左侧边缘偏移量的长度。

一个典型的位置值由两个值组成——第一个值水平地设置位置，第二个值垂直地设置位置。如果只指定一个轴的值，另一个轴将默认为 `center`。

7.5 字符串和标识符

如上面的颜色关键字以及伪元素 `::before` 和 `::after` 等

8. 排版

网页布局排版的美观，交互的友好与用户的留存息息相关，是前端需要掌握的重要内容之一，后续将从文字与布局两方面展开对排版中的样式属性的回顾与学习。

8.1 文字

CSS中有许多与文字相关的属性，比如颜色，字体大小，行高，文字阴影，字体等等，此处对其中常用属性展开回顾。

8.1.1 文字颜色

`color` 属性可以接收所有颜色的值类型，为元素设置字体颜色：

```
p {  
  color: red;  
}
```

除此之外，`color` 还会影响阴影的值，将在后方进行介绍

8.1.2 文字大小

在用户不改变浏览器默认字体大小的情况下，几乎所有浏览器中 `font-size` 的默认大小都是 `16px`

```
p {  
  font-size: 18px;  
}
```

但一般情况下我们不会通过固定大小 `px` 等单位对字体的大小进行控制，我们可以选择更灵活的属性如 `em`, `rem`, 百分比 等：

- `em` 让子元素可以通过父元素的 `font-size` 来进行计算获得新的字体大小，但在多级继承的情况下，比如 `div1>div2>div3`，我们让 `div2` 和 `div3` 均为 `1.5 em` 时，此时 `div2` 的大小是 `div1` 的 `1.5` 倍，`div3` 是 `div1` 的 `2.25` 倍，当元素过多的时候可能会出现我们意料之外的依赖关系。
- `rem` 则显得灵活得多，他以根元素为参考，在响应式布局中也起着重要的作用。若我们在设置字体或盒子大小的时候选择以 `rem` 为单位，当页面变动时，我们只需要改变 `html` 元素的 `font-size` 就能让页面的布局响应起来。

8.1.3 文字对齐

`text-align` CSS属性定义行内内容（例如文字）如何相对它的块父元素对齐。`text-align` 并不控制块元素自己的对齐，只控制它的行内内容的对齐。

值	说明
<code>left</code>	行内内容向左侧边对齐。
<code>right</code>	行内内容向右侧边对齐。
<code>center</code>	行内内容居中。
<code>justify</code>	文字向两侧对齐，对最后一行无效。
<code>justify-all</code>	和 <code>justify</code> 一致，但是强制使最后一行两端对齐。

CSS属性 `text-justify` 定义的是当文本的 `text-align` 属性被设置为 `:justify` 时的齐行方法。

`text-align-last` 控制最后一行的对齐方式，此处不再列举。

`text-indent` 控制缩进， 可以接收固定值和百分比， 通常缩进两个字的大小

```
p {  
  text-indent: 2em;  
}
```

8.1.4 文字间距

`letter-spacing` 属性用于设置文本字符的间距表现， `word-spacing` 用于声明标签和单词直接的间距行为， 两者值类型一致， 默认属性为**normal**,由字体本身以及浏览器默认大小来控制。

8.1.5 行高

`line-height` CSS 属性用于设置多行元素的空间量， 如多行文本的间距。对于块级元素， 它指定元素行盒的最小高度。对于非替代的 inline 元素， 它用于计算行盒的高度。

它可以接收一个数字得到的行高为该数字乘元素本省的font-size， 是推荐使用的类型， 一般为1.5， 一般浏览器默认为1.2左右， 但这也取决于元素的字体。

8.1.6 样式

`font-style` 属性允许你选择 font-family 字体下的 `italic` 或 `oblique` 样式。

用法：

值	说明
<code>normal</code>	选择常规字体
<code>italic</code>	选择斜体， 如果当前字体没有可用的斜体版本， 会选用倾斜体（ <code>oblique</code> ）替代。
<code>oblique</code>	选择倾斜体， 如果当前字体没有可用的倾斜体版本， 会选用斜体（ <code>italic</code> ）替代。

`font-weight` 属性指定了字体的粗细程度。

值	说明
<code>normal</code>	正常粗细。与 400 等值。
<code>bold</code>	加粗。与 700 等值。
<code>lighter</code>	比从父元素继承来的值更细(处在字体可行的粗细值范围内)。
<code>bolder</code>	比从父元素继承来的值更粗 (处在字体可行的粗细值范围内)
数字	一个介于 1 和 1000 (包含) 之间的数字类型值

`text-decoration` 这个 CSS 属性是用于设置文本的修饰线外观的（下划线、上划线、贯穿线/删除线 或 闪烁）它是 `text-decoration-line`, `text-decoration-color`, `text-decoration-style`, 和新出现的 `text-decoration-thickness` 属性的缩写。

具体样式可在网上查询

8.1.7 大小写转换

`text-transform` 属性指定如何将元素的文本大写。它可以用于使文本显示为全大写或全小写，也可单独对每一个单词进行操作。

值	说明
<code>capitalize</code>	这个关键字强制每个单词的首字母转换为大写，其他的字符保留不变。
<code>uppercase</code>	这个关键字强制所有字符被转换为大写。
<code>lowercase</code>	这个关键字强制所有字符被转换为小写。
<code>none</code>	这个关键字阻止所有字符的大小写被转换。

PS: `font-weight` 数值采取离散式定义（使用 100 的整倍数）。数值为实数，非 100 的整数倍的值将被四舍五入转换为 **100 的整倍数**，遇到 *50 时，将向上转换，如 150 将转换为 200。

8.1.8 阴影

在后续会展开讲解

8.2 盒子

网页的排版是由一个一个的盒子组成的，我们有很多的属性能装饰这些盒子：大小，背景，边框，阴影.....

8.2.1 宽与高

`width` 属性用于设置元素的宽度。`width` 默认设置**内容区域**的宽度，但如果 `box-sizing` 属性被设置为 `border-box`，就转而设置**边框区域**的宽度。

`min-width` 属性为给定元素设置最小宽度。它可以阻止 `width` 属性的应用值小于 `min-width` 的值。`min-width` 的值会同时覆盖 `max-width` 和 `width`。

`max-width` 会覆盖 `width` 设置, 但 `min-width` 设置会覆盖 `max-width`。

=> **优先级** `min-width` > `max-width` > `width`，块级盒子默认撑满了整行的宽度，此时加上了 `max-width` 或者 `min-width` 都会使其改变。

`height` 属性指定了一个元素的高度。默认情况下，这个属性决定的是**内容区**的高度，但是，如果将 `box-sizing` 设置为 `border-box`，这个属性决定的将是**边框区域**的高度。

`min-height` 能够设置元素的最小高度。这样能够防止 `height` 属性的应用值小于 `min-height` 的值。

`max-height` 设置元素的最大高度。它防止 `height` 属性的使用值大于 `max-height` 的指定值。

`max-height` 会覆盖 `height`，而 `min-height` 会覆盖 `max-height`。

优先级与宽度类似

思考：百分比单位参照的宽度是否包含 `padding` 或者 `border`？

8.2.2 内外边距

`padding` 简写属性控制元素所有四条边的内边距区域。

`padding` 属性接受 1~4 个值，取值不能为负。

- 当只指定一个值时，该值会统一应用到**全部四个边**的内边距上。
- 指定两个值时，第一个值会应用于**上边和下边**的内边距，第二个值应用于**左边和右边**。
- 指定三个值时，第一个值应用于**上边**，第二个值应用于**右边和左边**，第三个则应用于**下边**的内边距。
- 指定四个值时，依次（顺时针方向）作为**上边，右边，下边，和左边**的内边距。

`margin` 属性为给定元素设置所有四个（上下左右）方向的外边距属性。也就是 `margin-top`，`margin-right`，`margin-bottom`，和 `margin-left` 四个外边距属性设置的简写。

`margin` 属性接受 1~4 个值。取值为负时元素会比原来更接近临近元素。

- 当只指定一个值时，该值会统一应用到**全部四个边**的外边距上。
- 指定两个值时，第一个值会应用于**上边和下边**的外边距，第二个值应用于**左边和右边**。
- 指定三个值时，第一个值应用于**上边**，第二个值应用于**右边和左边**，第三个则应用于**下边**的外边距。
- 指定四个值时，依次（顺时针方向）作为**上边，右边，下边，和左边**的外边距。

8.2.3 背景与边框

背景

`background-color` 属性定义了CSS中任何元素的背景颜色。属性接受任何有效的 `color` 值。背景色扩展到元素的内容和内边距的下面。

`background-image` 属性允许在元素的背景中显示图像(可以引入多个，中间用逗号分隔)。默认情况下，`background-image` 所引入的图像会平铺满页面，其图像平铺的行为受到 `background-repeat` 属性控制，可用的值是：

值	说明
<code>no-repeat</code>	不重复
<code>repeat-x</code>	水平重复
<code>repeat-y</code>	垂直重复
<code>repeat</code>	在两个方向重复

单值语法是完整的双值语法的简写：

单值	等价于双值
<code>repeat-x</code>	<code>repeat no-repeat</code>
<code>repeat-y</code>	<code>no-repeat repeat</code>
<code>repeat</code>	<code>repeat repeat</code>

`background-size` 属性，它可以设置长度或百分比值，来调整图像的大小以适应背景。

- `cover` —浏览器将使图像足够大，使它完全覆盖了盒子区，同时仍然保持其高宽比。在这种情况下，有些图像可能会跳出盒子外

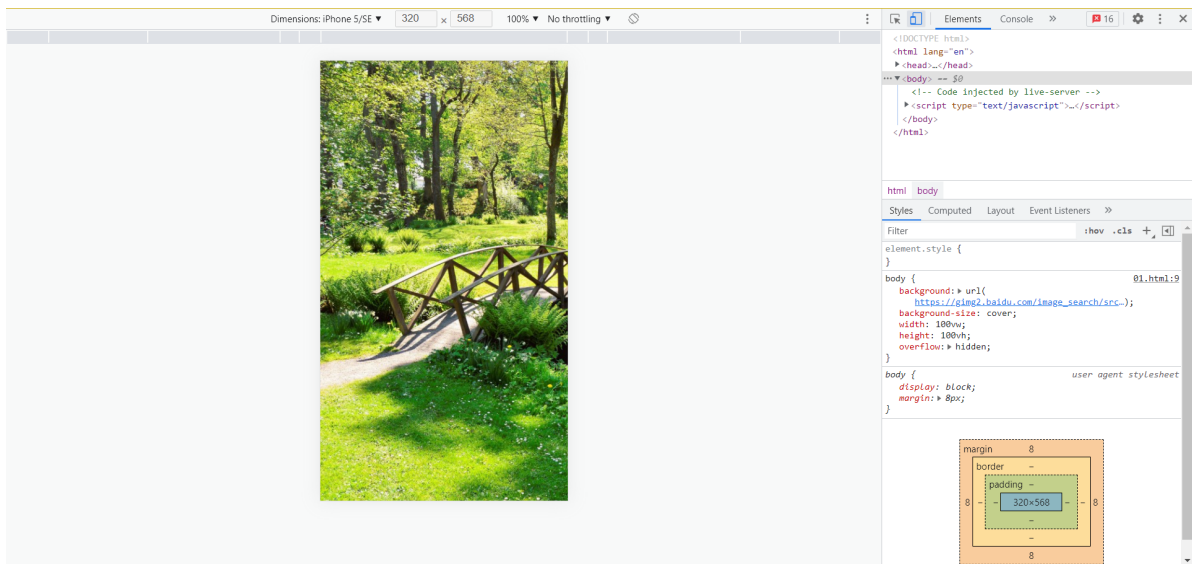
- **contain** — 浏览器将使图像的大小适合盒子内。在这种情况下，如果图像的长宽比与盒子的长宽比不同，则可能在图像的任何一边或顶部和底部出现间隙。

有时我们要让背景图片随着元素缩放，则需要使用百分比值。百分比值是相对于**容器**的大小而不是图片的大小，因此，简单的把图片的宽度和高度设置成百分比值，可能会因容器高度变化导致图片变形。

此时我们可以设置一个方向的使用百分比值，另一个为 **auto**，但百分比也不是任何情况下都适用，比如我们有一张宽屏的图片，在PC端我们向让他的宽度为100%。但在手机上宽度继续100%此时就会出现



这不是我们希望看到的，此时我们就可以使用cover属性，让浏览器决定哪边使用auto值，哪边使用100%



`background-position` 选择背景图像显示在其应用到的盒子中的位置。它使用的坐标系中，框的左上角是(0,0)，框沿着水平(x)和垂直(y)轴定位。

注意：默认的背景位置值是(0,0)，`background-position` 是 `background-position-x` 和 `background-position-y` 的简写。`background-position` 接受关键字 `top`, `left`, `right`, `bottom`, `center`，也接受长度值和百分比。

最后，还可以使用4-value语法来指示到盒子的某些边的距离

```
body {  
  background: url('imgxxx');  
  background-size: 200px;  
  background-repeat: no-repeat;  
  background-position: top 20px left 20px;  
  width: 100vw;  
  height: 100vh;  
  overflow: hidden;  
}
```



`background-attachment` 属性决定背景图像的位置是在视口内固定，或者随着包含它的区块滚动。属性如下：

值	说明
<code>fixed</code>	此关键属性值表示背景相对于视口固定。即使一个元素拥有滚动机制，背景也不会随着元素的内容滚动。
<code>local</code>	此关键属性值表示背景相对于元素的内容固定。如果一个元素拥有滚动机制，背景将会随着元素的内容滚动，并且背景的绘制区域和定位区域是相对于可滚动的区域而不是包含他们的边框。
<code>scroll</code>	此关键属性值表示背景相对于元素本身固定，而不是随着它的内容滚动（对元素边框是有效的）。

`background-clip` 设置元素的背景（背景图片或颜色）是否延伸到边框、内边距盒子、内容盒子下面。

值	说明
<code>border-box</code>	背景延伸至边框外沿（但是在边框下层）。
<code>padding-box</code>	背景延伸至内边距（padding）外沿。不会绘制到边框处。
<code>content-box</code>	背景被裁剪至内容区（content box）外沿。

边框

我们通常使用的是border这个属性来定义边框的样式。例如

```
border: 1px solid #ccc;
```

这是三个属性 `border-width`，`border-style`，`border-color` 的缩写, 此处我们简单介绍一下这几个属性:

`border-width` 属性可以设置盒子模型的边框宽度，同时类似于内外边距的写法，`border-width` 也分别包含了 `border-top-width`，`border-bottom-width`，和 `border-left-width`，`border-color` 也是如此，我们都可以分别设置四条边的值。

`border-style` 用来设定元素所有边框的样式。

注意: `border-style` 默认值是 `none`，这意味着如果只修改 `border-width` 和 `border-color` 是**不会出现**边框的。

下面给出几种常见的样式：

值	说明
<code>none</code>	和关键字 <code>hidden</code> 类似，不显示边框。在这种情况下，如果没有设定背景图片， <code>border-width</code> 计算后的值将是 0，即使先前已经指定过它的值。在单元格边框重叠情况下， <code>none</code> 值优先级最低，意味着如果存在其他的重叠边框，则会显示为那个边框。
<code>dashed</code>	显示为一系列短的方形虚线。标准中没有定义线段的长度和大小，视不同实现而定。
<code>solid</code>	显示为一条实线。
<code>dotted</code>	显示为一系列圆点。标准中没有定义两点之间的间隔大小，视不同实现而定。圆点半径是 <code>border-width</code> 计算值的一半。

`border-radius` 允许你设置元素的外边框圆角。当使用一个半径时确定一个圆形，当使用两个半径时确定一个椭圆。这个(椭圆)与边框的交集形成圆角效果。它也是四个方位的简写，可以分开来写，此处不再展开。

此处列出几个需要注意的点：

- 即使元素没有边框，圆角也可以用到 `background` 上面，具体效果受 `background-clip` 影响。
- 当 `border-collapse` 的值为 `collapse` 时，`border-radius` 属性不会被应用到表格（

）元素上。

`border-collapse` 属性是用来决定表格的边框是分开的还是合并的。在分隔模式下，相邻的单元格都拥有独立的边框。在合并模式下，相邻单元格共享边框。

- 合并（collapsed）模式下，表格中相邻单元格共享边框。在这种模式下，CSS属性 `border-style` 的值 `inset` 表现为槽，值 `outset` 表现为脊。
- 分隔（separated）模式是 HTML 表格的传统模式。相邻单元格都拥有不同的边框。边框之间的距离是通过属性 `border-spacing` 来确定的。

8.3 阴影

此处的阴影将从 `text-shadow` 和 `box-shadow` 展开，`text-shadow` 为文字添加阴影。可以为文字与 `text-decoration` 添加多个阴影，阴影值之间用逗号隔开。每个阴影值由元素在X和Y方向的偏移量、模糊半径和颜色值组成。`box-shadow` 属性用于在元素的框架上添加阴影效果。你可以在同一个元素上设置多个阴影效果，并用逗号将他们分隔开。该属性可设置的值包括阴影的X轴偏移量、Y轴偏移量、模糊半径、扩散半径和颜色。

向元素添加单个 `box-shadow` 效果时使用以下规则：

当给出两个、三个或四个值时。

- 如果只给出两个值，那么这两个值将会被当水平偏移和垂直偏移。
- 如果给出了第三个值，值越大，模糊面积越大，阴影就越大越淡。**不能为负值**。默认为0，此时阴影边缘锐利。。
- 如果给出了第四个值，取正值时，阴影扩大；取负值时，阴影收缩。默认为0，此时阴影与元素同样大。
- 可选，`inset`关键字，此时阴影会在边框之内（即使是透明边框）、背景之上、内容之下。
- 可选，阴影颜色值。

`text-shadow` 用法与上文类似，可自主尝试。

9. 布局

本节将从定位 `position`，弹性盒子 `flex` 和栅格系统 `grid` 来展开对布局相关知识点的梳理。

9.1 定位 POSITION

`position` 属性用于指定一个元素在文档中的定位方式。`top`，`right`，`bottom` 和 `left` 属性则决定了该元素的最终位置。定位元素（positioned element）是其计算后位置属性为 `relative`，`absolute`，`fixed` 或 `sticky` 的一个元素（换句话说，除`static`以外的任何东西）。相对定位的元素并未脱离文档流，而绝对定位的元素则脱离了文档流。在布置文档流中其它元素时，绝对定位元素不占据空间。绝对定位元素相对于最近的非 `static` 祖先元素定位。

`static` 关键字指定元素使用正常的布局行为，即元素在文档常规流中当前的布局位置。此时 `top`，`right`，`bottom` 和 `left` 和 `z-index` 属性无效。

定位的参照是什么 [布局和包含块](#)

9.1.1 相对定位

`relative`

该关键字下，元素先放置在未添加定位时的位置，再在不改变页面布局的前提下调整元素位置（因此会在此元素未添加定位时所在位置留下空白）。`position: relative` 对 `table-group`, `table-row`, `table-column`, `table-cell`, `table-caption` 元素无效

9.1.2 绝对定位

`absolute`

元素会被移出正常文档流，并不为元素预留空间，通过指定元素相对于最近的非 `static` 定位祖先元素的偏移，来确定元素位置。绝对定位的元素可以设置外边距，且不会与其他边距合并。

9.1.3 固定定位

`fixed`

元素会被移出正常文档流，并不为元素预留空间，而是通过指定元素相对于屏幕视口（viewport）的位置来指定元素位置。元素的位置在屏幕滚动时不会改变。打印时，元素会出现在的每页的固定位置。

`fixed` 属性会创建新的层叠上下文。当元素祖先的 `transform`, `perspective` 或 `filter` 属性非 `none` 时，容器由视口改为该祖先。

9.1.4 粘性定位

`sticky`

元素根据正常文档流进行定位，然后相对它的最近滚动祖先（nearest scrolling ancestor）和最近块级祖先，包括 `table-related` 元素，基于 `top`, `right`, `bottom`, 和 `left` 的值进行偏移。偏移值不会影响任何其他元素的位置。

该值总是创建一个新的层叠上下文（stacking context）。注意，一个 `sticky` 元素会“固定”在离它最近的一个拥有“滚动机制”的祖先上（当该祖先的 `overflow` 是 `hidden`, `scroll`, `auto`, 或 `overlay` 时），即便这个祖先不是最近的真实可滚动祖先。

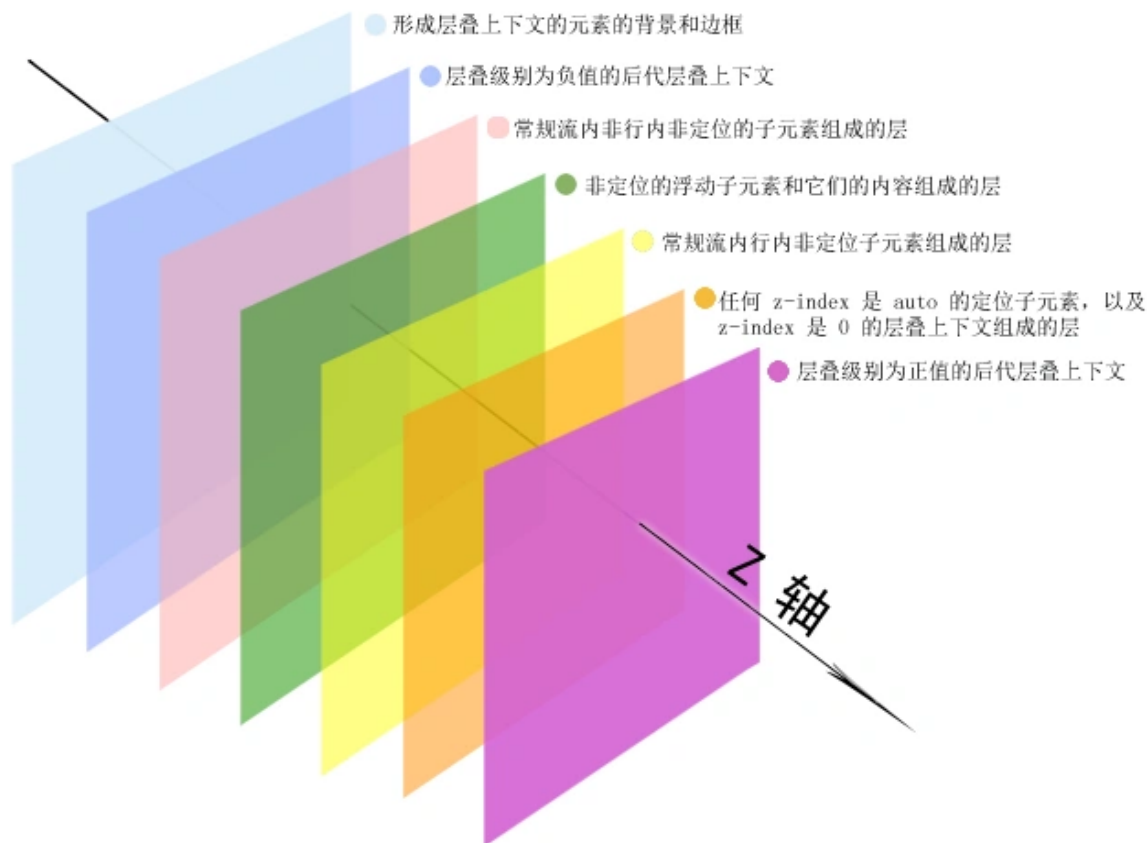
粘性定位可以被认为是**相对定位**和**固定定位**的混合。元素在跨越特定阈值前为相对定位，之后为固定定位。须指定 `top`, `right`, `bottom` 或 `left` 四个阈值其中之一，才可使粘性定位生效。否则其行为与相对定位相同。

9.1.5 z-index

`z-index` 属性设定了一个定位元素及其后代元素或 flex 项目的 z-order。当元素之间重叠的时候，`z-index` 较大的元素会覆盖较小的元素在上层进行显示。对于一个已经定位的盒子（即其 `position` 属性值不是 `static`，这里要注意的是 CSS 把元素看作盒子），`z-index` 属性指定：

1. 盒子在**当前层叠上下文**中的层叠层级。
2. 盒子是否创建一个**本地层叠上下文**。

9.1.6 层叠上下文



我们假定用户正面向（浏览器）视窗或网页，而 HTML 元素沿着其相对于用户的一条虚构的 `z` 轴排开，**层叠上下文**就是对这些 HTML 元素的一个三维构想。众 HTML 元素基于其元素属性按照优先级顺序占据这个空间。

文档中的层叠上下文由满足以下任意一个条件的元素形成：

- 文档根元素 `html`；
- `position` 值为 `absolute` 或 `relative` 且 `z-index` 值不为 `auto` 的元素；
- `position` 值为 `fixed` 或 `sticky` 的元素；
- flex容器的子元素，且 `z-index` 值不为 `auto`；
- grid容器的子元素，且 `z-index` 值不为 `auto`；
- `opacity` 属性值小于 `1` 的元素；
- `mix-blend-mode` 属性值不为 `normal` 的元素；
- 以下任意属性值不为 `none` 的元素：
 - `transform`
 - `filter`
 - `perspective`
 - `clip-path`
 - `mask` / `mask-image` / `mask-border`
- `isolation` 属性值为 `isolate` 的元素；
- `-webkit-overflow-scrolling` 属性值为 `touch` 的元素；
- `will-change` 值设定了任一属性而该属性在 `non-initial` 值时会创建层叠上下文的元素；
- `contain` 属性值为 `layout`、`paint` 或包含它们其中之一的合成值（比如 `contain: strict`、`contain: content`）的元素。

在层叠上下文中，子元素同样也按照上面解释的规则进行层叠。重要的是，其子级层叠上下文的 `z-index` 值只在父级中才有意义。子级层叠上下文被自动视为父级层叠上下文的一个独立单元。

总结:

- 层叠上下文可以包含在其他层叠上下文中，并且一起创建一个层叠上下文的层级。
- 每个层叠上下文都完全独立于它的兄弟元素：当处理层叠时只考虑子元素。
- 每个层叠上下文都是自包含的：当一个元素的内容发生层叠后，该元素将被作为整体在父级层叠上下文中按顺序进行层叠。

9.2 弹性盒子 FLEX

CSS 弹性盒子布局是 CSS 的模块之一，定义了一种针对用户界面设计而优化的 CSS 盒子模型。在弹性布局模型中，弹性容器的子元素可以在任何方向上排布，也可以“弹性伸缩”其尺寸，既可以增加尺寸以填满未使用的空间，也可以收缩尺寸以避免父元素溢出。子元素的水平对齐和垂直对齐都能很方便的进行操控。通过嵌套这些框（水平框在垂直框内，或垂直框在水平框内）可以在两个维度上构建布局。

flex 容器中的所有 flex 元素都会有下列行为：

- 元素排列为一行（`flex-direction` 属性的初始值是 `row`）。
- 元素从主轴的起始线开始。
- 元素不会在主维度方向拉伸，但是可以缩小。
- 元素被拉伸来填充交叉轴大小。
- `flex-basis` 属性为 `auto`。
- `flex-wrap` 属性为 `nowrap`。

这会让你的元素呈线形排列，并且把自己的大小作为主轴上的大小。如果有太多元素超出容器，它们会溢出而不会换行。如果一些元素比其他元素高，那么元素会沿交叉轴被拉伸来填满它的大小。

9.2.1 方向

`flex-direction` 属性指定了内部元素是如何在 flex 容器中布局的，定义了主轴的方向(正方向或反方向)。

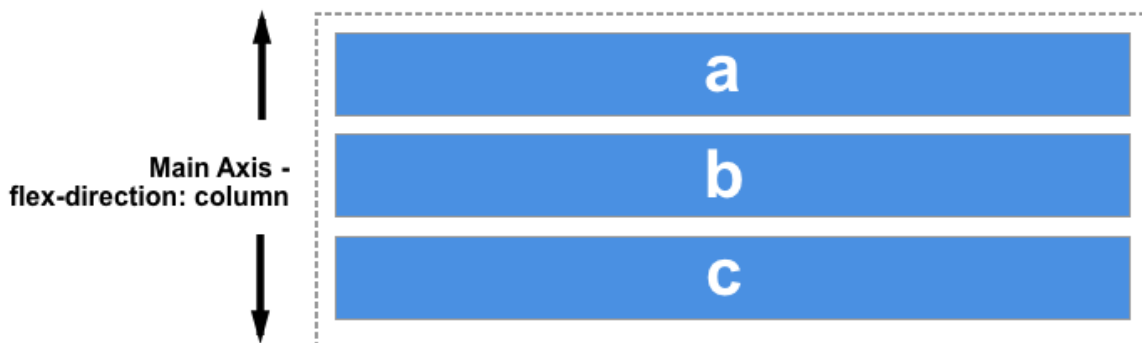
当使用 flex 布局时，主轴由 `flex-direction` 定义，另一根轴垂直于它。我们使用 flexbox 的所有属性都跟这两根轴线有关，我们先来回顾一下主轴和交叉轴的概念。

主轴由 `flex-direction` 定义，可以取4个值：

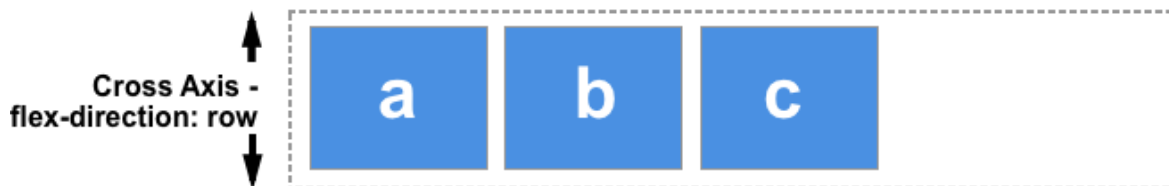
值	说明
<code>row</code>	flex容器的主轴被定义为与文本方向相同。 主轴起点和主轴终点与内容方向相同。
<code>row-reverse</code>	表现和row相同，但是置换了主轴起点和主轴终点
<code>column</code>	flex容器的主轴和块轴相同。主轴起点与主轴终点和书写模式的前后点相同
<code>column-reverse</code>	表现和 column 相同，但是置换了主轴起点和主轴终点



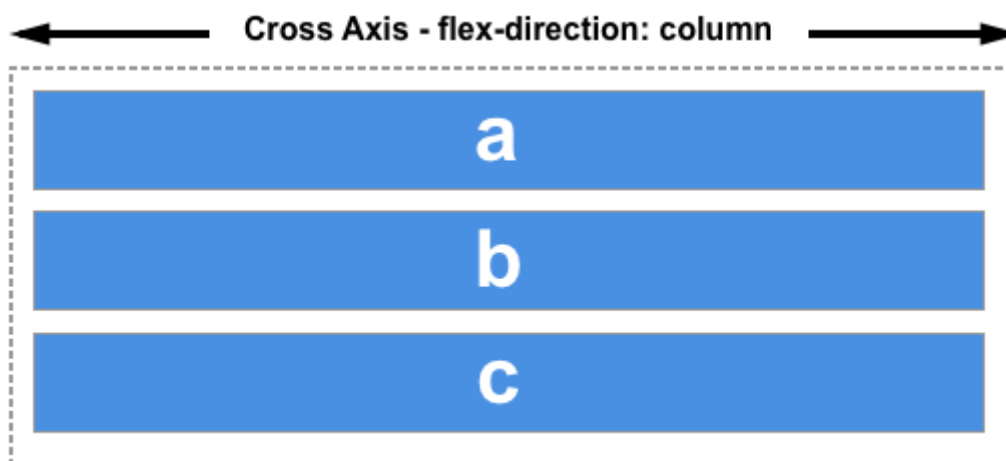
选择 `column` 或者 `column-reverse` 时，你的主轴会沿着上下方向延伸 — 也就是 **block 排列** 的方向。



交叉轴垂直于主轴，所以如果你的 `flex-direction` (主轴) 设成了 `row` 或者 `row-reverse` 的话，交叉轴的方向就是沿着列向下的。



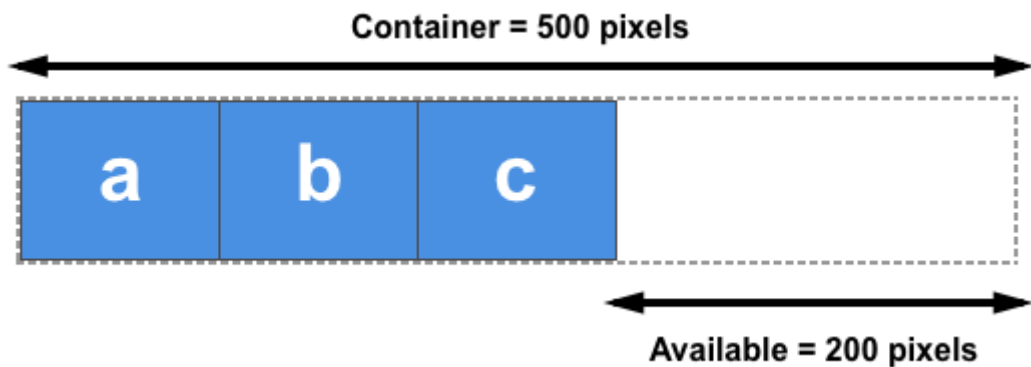
如果主轴方向设成了 `column` 或者 `column-reverse`，交叉轴就是水平方向。



理解主轴和交叉轴的概念对于对齐 flexbox 里面的元素是很重要的；flexbox 的特性是沿着主轴或者交叉轴对齐之中的元素。

在了解下面的三个属性之前，我们先了解一下**可用空间**的概念

假设在 1 个 500px 的容器中，我们有 3 个 100px 宽的元素，那么这 3 个元素需要占 300px 的宽，剩下 200px 的可用空间。在默认情况下，flexbox 的行为会把这 200px 的空间留在最后一个元素的后面。



我们可用通过 `flex-grow`, `flex-shrink`, `flex-basis` 属性去控制剩下的空间

9.2.2 增长系数

`flex-grow` 若被赋值为一个正整数，flex 元素会以 `flex-basis` 为基础，沿主轴方向增长尺寸。这会使该元素延展，并占据此方向轴上的可用空间（available space）。如果有其他元素也被允许延展，那么他们会各自占据可用空间的一部分。

如果我们给上例中的所有元素设定 `flex-grow` 值为1，容器中的可用空间会被这些元素平分。它们会延展以填满容器主轴方向上的空间。

`flex-grow` 属性可以按比例分配空间。如果第一个元素 `flex-grow` 值为2，其他元素值为1，则第一个元素将占有2/4（上例中，即为 200px 中的 100px），另外两个元素各占有1/4（各50px）。

如果我们想要完全平分三个属性的时候，我们可以把 `flex-basis` 和元素自生的宽度设置为0，然后通过把 `flex-grow` 设置为1，让他们平均分配所有的空间。

```
<style>
  section {
    display: flex;
    width: 600px;
    background-color: red;
  }

  div {
    flex: 1;
    width: 0;
    flex-basis: 0;
    border: 1px solid #000;
  }
</style>
<section>
  <div>1</div>
  <div>2</div>
  <div>3</div>
</section>
```



9.2.3 缩小系数

`flex-grow` 属性是处理flex元素在主轴上增加空间的问题，相反 `flex-shrink` 属性是处理flex元素收缩的问题。如果我们的容器中没有足够排列flex元素的空间，那么可以把flex元素 `flex-shrink` 属性设置为正整数来缩小它所占空间到 `flex-basis` 以下。

与 `flex-grow` 属性一样，可以赋予不同的值来控制flex元素收缩的程度——给 `flex-shrink` 属性赋予更大的数值可以比赋予小数值的同级元素收缩程度更大。在计算flex元素收缩的大小时，它的最小尺寸也会被考虑进去，就是说实际上flex-shrink属性可能会和flex-grow属性表现的不一致。

在计算flex元素收缩的大小时，它的最小尺寸也会被考虑进去，就是说实际上flex-shrink属性可能会和flex-grow属性表现的不一致。因此，我们可以在文章《[控制Flex子元素在主轴上的比例](#)》中更详细地看一下这个算法的原理。

我们未必一定要记住上述这个算法的原理，但是了解有助于我们在遇到类似问题的时候能进行快速定位

9.2.4 空间大小

`flex-basis` 指定了 flex 元素在主轴方向上的初始大小。如果不使用 `box-sizing` 改变盒模型的话，那么这个属性就决定了 flex 元素的内容盒 (content-box) 的尺寸。

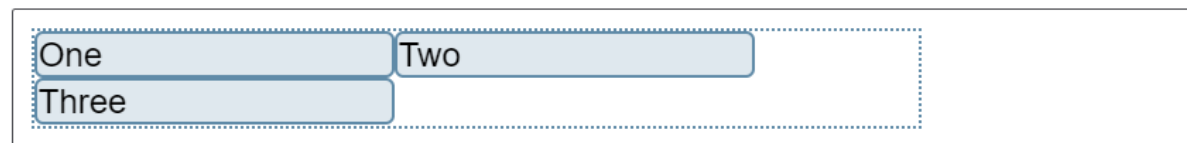
注意: 当一个元素同时被设置了 `flex-basis` (除值为 `auto` 外) 和 `width` (或者在 `flex-direction: column` 情况下设置了`height`) , `flex-basis` 具有更高的优先级。

`flex-basis` 属性在任何空间分配发生之前初始化flex子元素的尺寸. 此属性的初始值为 `auto`. 如果 `flex-basis` 设置为 `auto` , 浏览器会先检查flex子元素的主尺寸是否设置了绝对值再计算出flex子元素的初始值. 比如说你已经给你的flex子元素设置了200px 的宽, 则200px 就是这个flex子元素的 `flex-basis` .

9.2.5 多行容器

`flex-wrap` 指定 flex 元素单行显示还是多行显示。如果允许换行，这个属性允许我们控制行的堆叠方向。

虽然 `flexbox` 是一维模型，但可以使我们的 `flex` 项目应用到多行中。在这样做的时候，我们应该把每一行看作一个新的 `flex` 容器。任何空间分布都将在该行上发生，而不影响该空间分布的其他行。



`flex-wrap` 的值设置为 `wrap` , 项目的子元素换行显示。若将其设置为 `nowrap` , 这也是初始值, 它们将会缩小以适应容器, 因为它们使用的是允许缩小的初始 `Flexbox` 值。如果项目的子元素无法缩小, 使用 `nowrap` 会导致溢出, 或者缩小程度还不够小。

其他: `flex-flow` 属性是 `flex-direction` 和 `flex-wrap` 的简写。

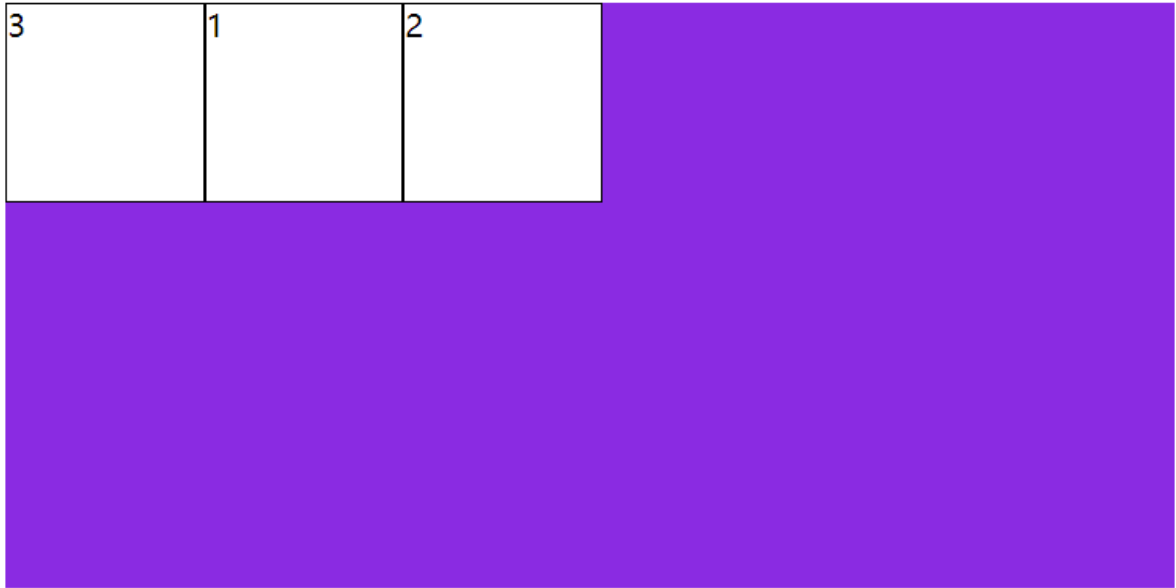
9.2.6 order

`order` 属性规定了弹性容器中的可伸缩项目在布局时的顺序。元素按照 `order` 属性的值的增序进行布局。拥有相同 `order` 属性值的元素按照它们在源代码中出现的顺序进行布局。

示例:

```
div {
  width: 100px;
  height: 100px;
  background-color: #fff;
  border: 1px solid #000;
  color: #000;
  order: 3;
}

div:last-child {
  order: 1;
}
```

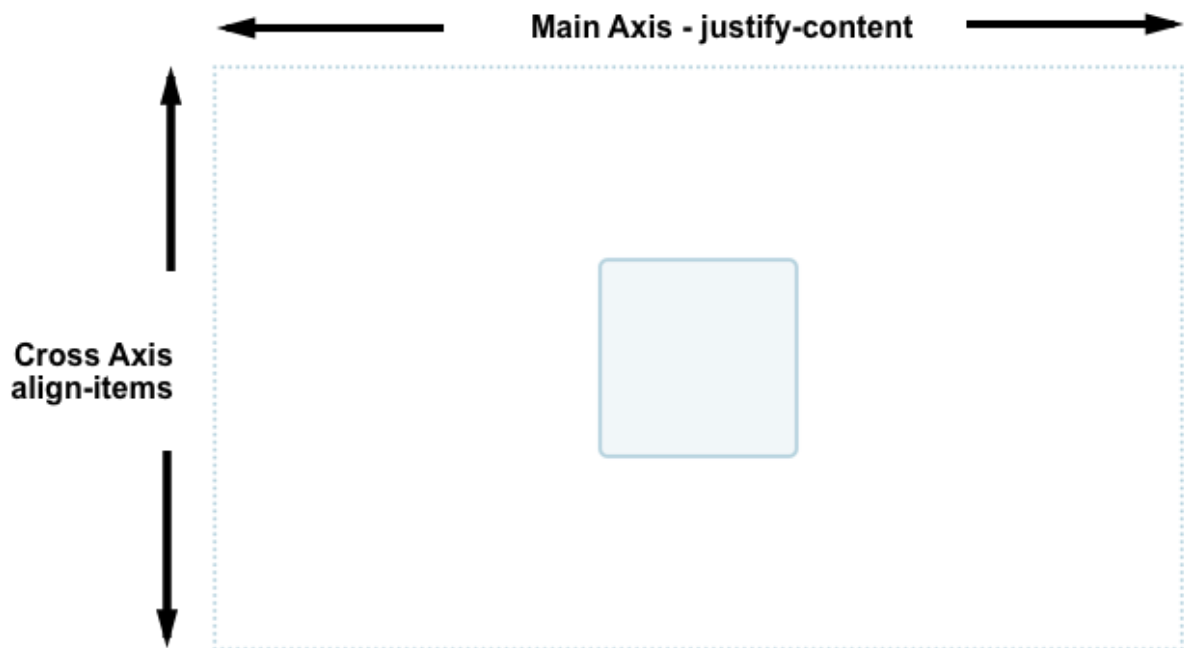


我们可以注意到3号盒子排到了最前面

9.2.7 对齐方式

flexbox之所以能迅速吸引开发者的注意，其中一个原因就是它首次为网页样式居中提供了合适的方案。得益于它提供的合适的垂直居中能力，我们可以很轻松地把一个盒子居中。在这份指南里，我们将详细地介绍flexbox的垂直和水平居中的工作原理。

为了使我们的盒子居中，通过 `align-items` 属性，可以将交叉轴上的item对齐，此时使用的是垂直方向的块轴。而使用 `justify-content` 则可以对齐主轴上的项目，主轴是水平方向的。

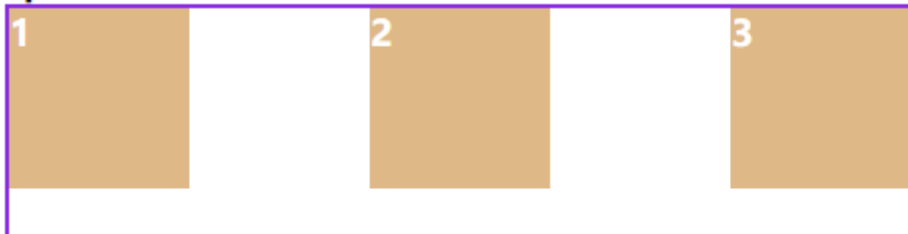


主轴对齐

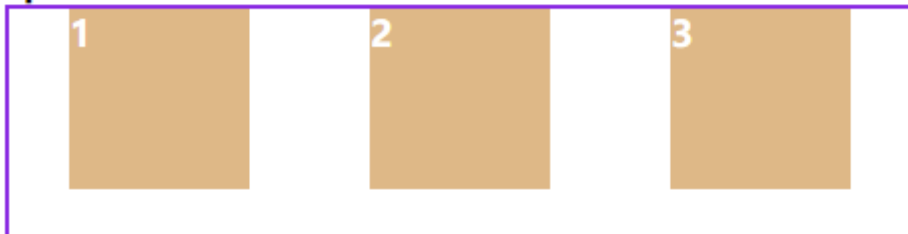
`justify-content` 属性定义了浏览器之间，如何分配顺着弹性容器主轴(或者网格行轴) 的元素之间及其周围的空间。

值	说明
<code>flex-start</code>	从行首开始排列。每行第一个弹性元素与行首对齐，同时所有后续的弹性元素与前一个对齐。
<code>flex-end</code>	从行尾开始排列。每行最后一个弹性元素与行尾对齐，其他元素将与后一个对齐。
<code>center</code>	伸缩元素向每行中点排列。每行第一个元素到行首的距离将与每行最后一个元素到行尾的距离相同。
<code>space-between</code>	在每行上均匀分配弹性元素。相邻元素间距离相同。每行第一个元素与行首对齐，每行最后一个元素与行尾对齐。
<code>space-around</code>	在每行上均匀分配弹性元素。相邻元素间距离相同。每行第一个元素到行首的距离和每行最后一个元素到行尾的距离将会是相邻元素之间距离的一半。
<code>space-evenly</code>	flex项都沿着主轴均匀分布在指定的对齐容器中。相邻flex项之间的间距，主轴起始位置到第一个flex项的间距，主轴结束位置到最后一个flex项的间距，都完全一样。

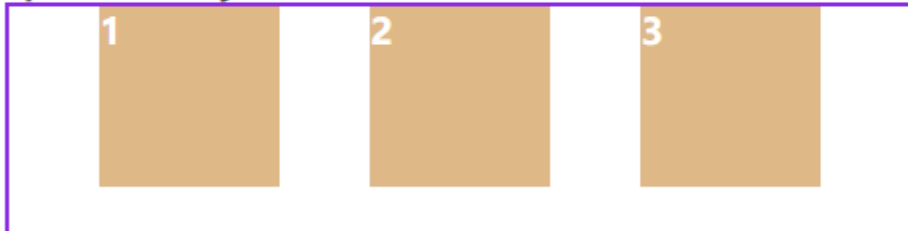
space-between



space-around



space-evenly



交叉轴对齐

`align-items` 属性将所有直接子节点上的 `align-self` 值设置为一个组。`align-self` 属性设置项目在其包含块中在交叉轴方向上的对齐方式。

值	说明
<code>flex-start</code>	元素向侧轴起点对齐。
<code>flex-end</code>	元素向侧轴终点对齐。
<code>center</code>	元素在侧轴居中。如果元素在侧轴上的高度高于其容器，那么在两个方向上溢出距离相同。

`align-self` - 控制交叉轴（纵轴）上的单个 flex 项目的对齐。

`align-items` 属性是给所有 flex 项目统一设置 `align-self` 的对齐属性。这意味着你能给单个 flex 项目明确地声明 `align-self` 属性。`align-self` 拥有 `align-items` 的所有属性值，另外还有一个 `auto` 能重置自身的值为 `align-items` 定义的值。

若出现多行的情况，我们需要控制多行的间距时可以使用 `align-content` 进行控制，其属性与 `justify-content` 的属性类似。

备注：`justify-self` 和 `justify-items` 在 flex 布局中是被忽略的

9.3 栅格系统 GRID

CSS 网格布局擅长于将一个页面划分为几个主要区域，以及定义这些区域的大小、位置、层次等关系（前提是HTML生成了这些区域）。像表格一样，网格布局让我们能够按行或列来对齐元素。然而在布局上，网格比表格更可能做到或更简单。例如，网格容器的子元素可以自己定位，以便它们像CSS定位的元素一样，真正的有重叠和层次。

栅格系统的属性特别多，但用法都是相通的。

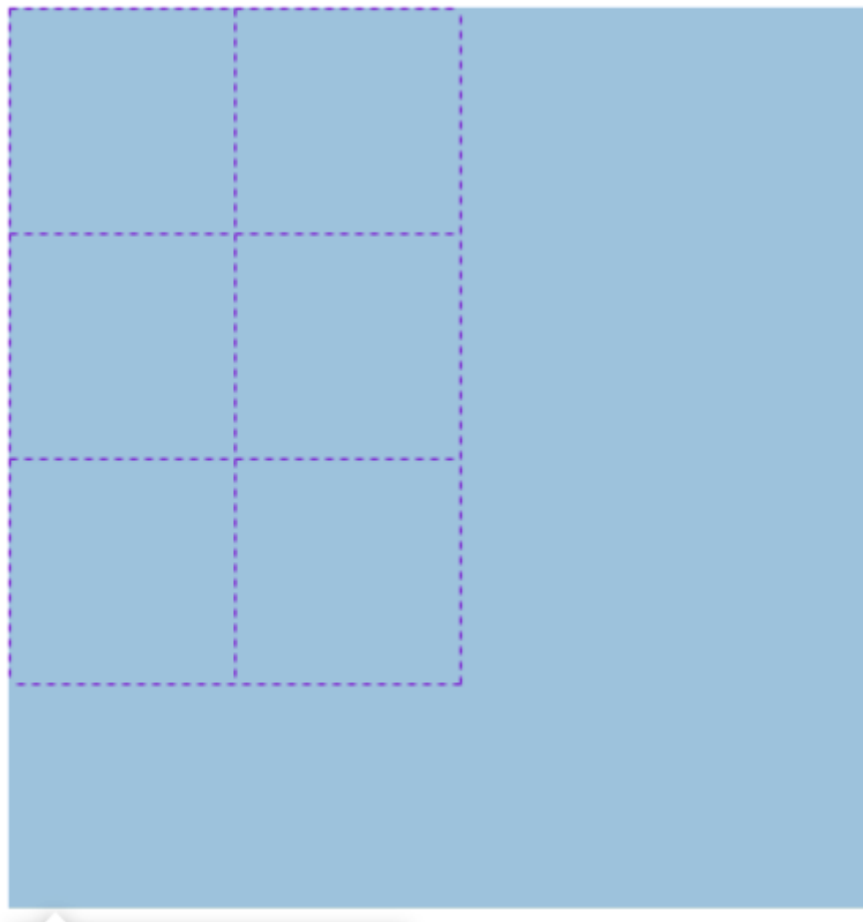
9.3.1 网格模板

`grid-template-rows` 该属性是基于 网格行 的维度，去定义网格线的名称和网格轨道的尺寸大小。

`grid-template-columns` 该属性是基于 网格列 的维度，去定义网格线的名称和网格轨道的尺寸大小，下面通过一个小例子来说明基本使用。

首先我们定义一个**三行两列**的表格

```
section {
  display: grid;
  width: 400px;
  height: 400px;
  background-color: beige;
  grid-template-columns: 100px 100px;
  grid-template-rows: 100px 100px 100px;
}
```



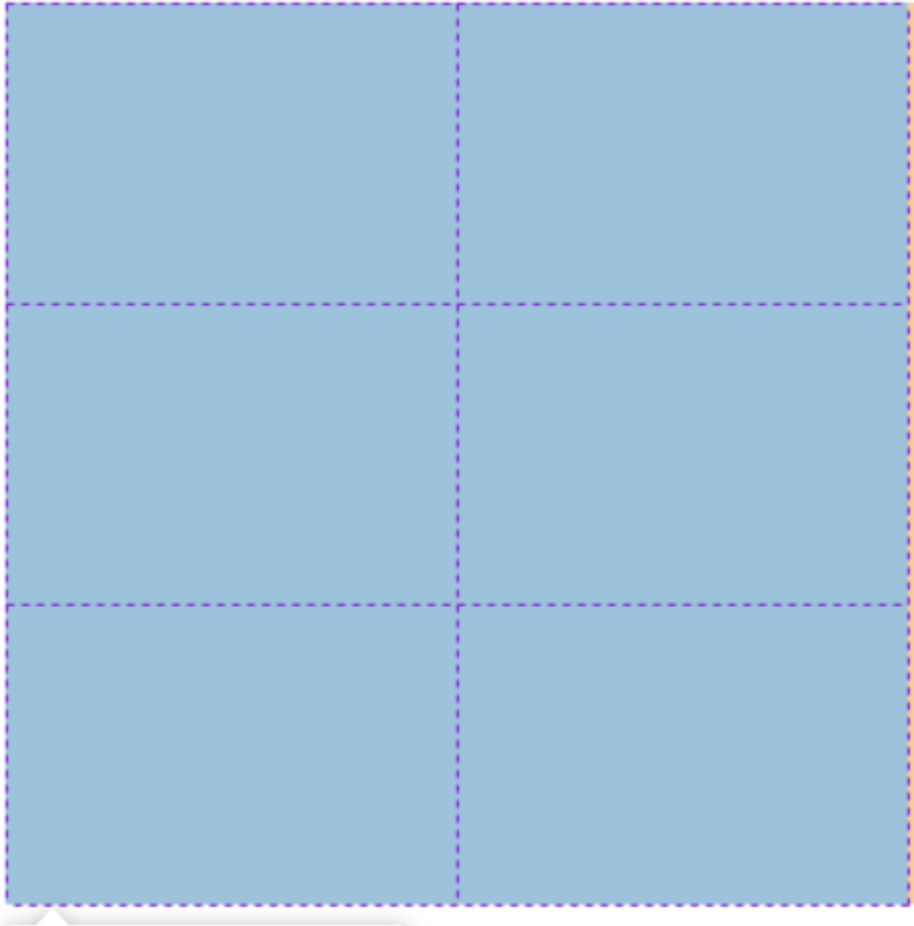
我们发现网格的大小是我们写入的固定值 `100px`，我们想让他自适应可以使用百分比。但有个更方便的单位 `fr` 类似于弹性布局中的 `flex-grow`，我们可以通过指定**份数**来控制他们的自适应。

```
grid-template-columns: 1fr 1fr;  
grid-template-rows: 1fr 1fr 1fr;
```

通过函数repeat我们可以进一步简写

```
grid-template-columns: repeat(2, 1fr);  
grid-template-rows: repeat(3, 1fr);
```

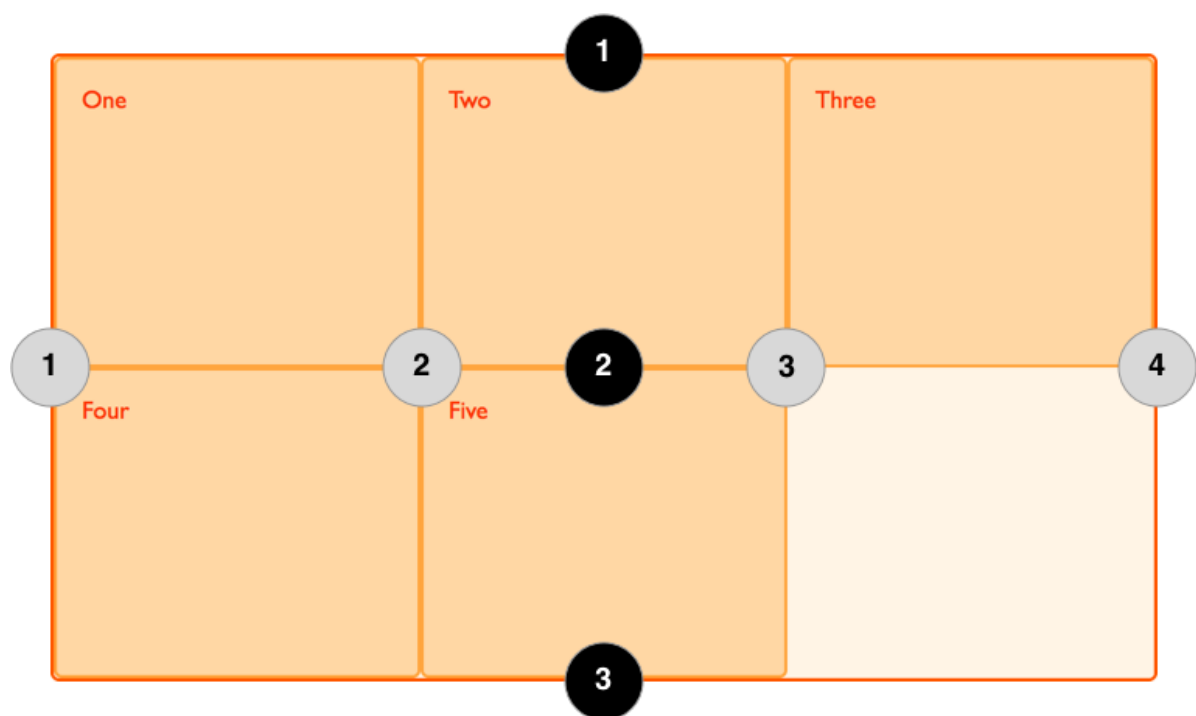
此时我们可以发现块元素的所有空间均被分配



既然分配了空间，接下来就是根据需求去使用了，我们可以通过 `grid-row-start`, `grid-row-end`, `grid-column-start`, `grid-column-end` 去使用这些空间，下一节将具体说明。

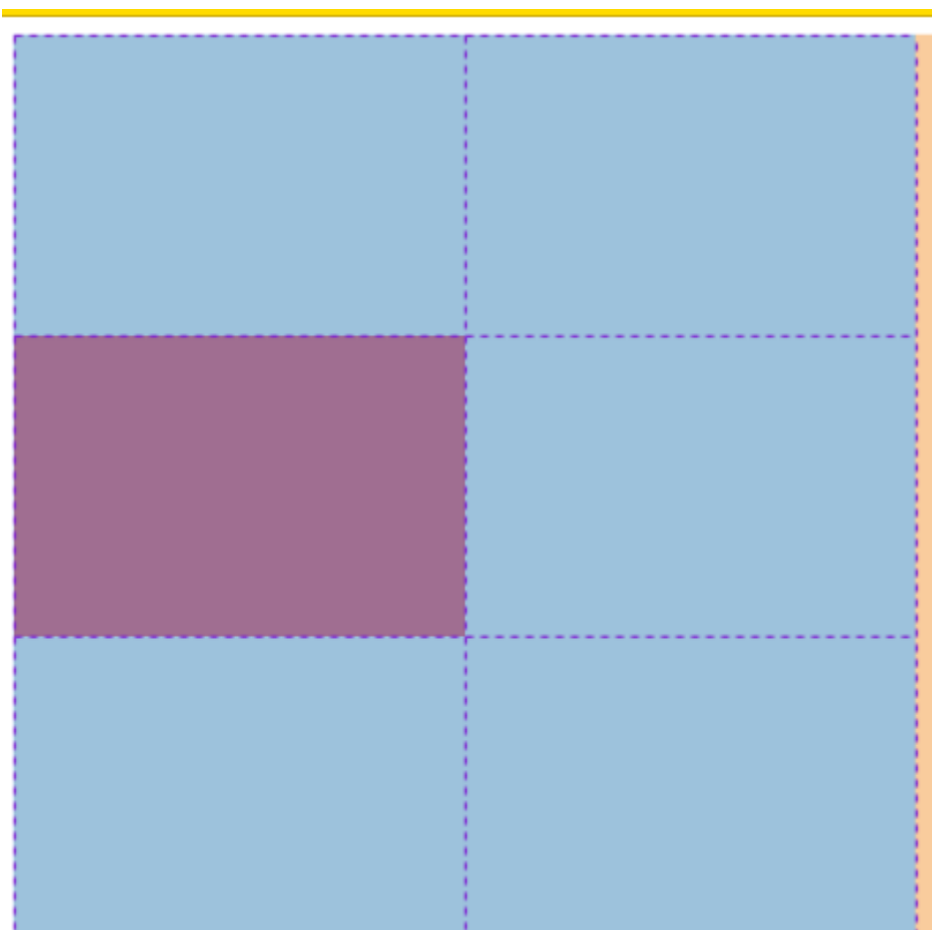
9.3.2 网格线

使用Grid布局在显式网格中定义轨道的同时会创建网格线,网格线可以用它们的编号来寻址。



还是之前的例子，此时我们要指定子元素到某一个网格中，我们可以给他们指定上一节的四个属性

```
div {
  grid-row-start: 2;
  grid-row-end: 3;
  grid-column-start: 1;
  grid-column-end: 2;
  background-color: red;
}
```

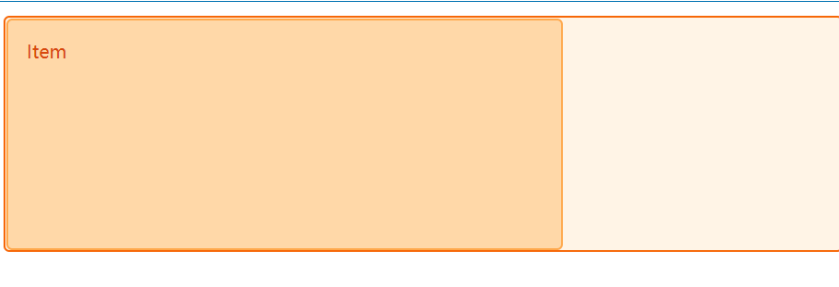


如果只指定了开始或只制定了结束位置的时候，默认占一个单元格。

`div` 元素预期占用了我们期望的位置，数字定义网格线的位置十分方便，与此同时我们还可以为网格线命名，这在某些情境下会更高效。

此处引用MDN上面的例子

```
.wrapper {
  display: grid;
  grid-template-columns: [col1-start] 1fr [col2-start] 1fr [col3-start] 1fr [cols-end];
  grid-template-rows: [row1-start] 100px [row2-start] 100px [rows-end];
}
.item {
  grid-column-start: col1-start;
  grid-column-end: col3-start;
  grid-row-start: row1-start;
  grid-row-end: rows-end;
}
```



属性简写：

`grid-row` 和 `grid-column`，他们可以同时接受起始和结束位置，中间用 `/` 相隔

```
div {
  grid-row: 2 / 3;
  grid-column: 1 / 2;
}
```

当我们有确定的起始位置，可以用一个更方便的方式自动计算结束位置，`span` 关键字为网格单元定义一个**跨度**，使得网格单元的网格区域中的一条边界远离另一条边界线 `n` 条基线。

```
div {
  grid-row: 2 / span 1;
  grid-column: 1 / span 1;
}
```

还可以简写为 `grid-area`，话不多说，上代码

```
div {
  grid-area: 2 / 1 / span 1 / span 1;
}
```

如果指定了4个 `<grid-line>` 的值，`grid-row-start` 会被设为第一个值，`grid-column-start` 为第二个值，`grid-row-end` 为第三个值，`grid-column-end` 为第四个值。

9.3.3 网格区域

`grid-template-areas` CSS 属性是网格区域 grid areas 在CSS中的特定命名。

此时我们有三个元素a,b,c。每一个给定的字符串会生成一行，一个字符串中用空格分隔的每一个单元 (cell)会生成一列。多个同名的，跨越相邻行或列的单元称为网格区块(grid area)。非矩形的网格区块是无效的。

下面依然给出MDN上的例子

The image displays three examples of CSS grid layouts, each with a code editor on the left and a visual representation on the right.

Example 1:

```
grid-template-areas:  
  "a a a"  
  "b c c"  
  "b c c";
```

The visual shows a grid where 'One (a)' (blue) is at the top, and 'Two (b)' (pink) and 'Three (c)' (green) are side-by-side below it.

Example 2:

```
grid-template-areas:  
  "a a a"  
  "b c c"  
  "b c c";
```

The visual shows 'Two (b)' (pink) on the left, 'One (a)' (blue) at the top right, and 'Three (c)' (green) at the bottom right.

Example 3:

```
grid-template-areas:  
  "a a a"  
  "b c c"  
  "b c c";
```

The visual shows 'One (a)' (blue) at the top left, and 'Two (b)' (pink) and 'Three (c)' (green) side-by-side at the bottom.

在最后一个模板中我们看见 `.` 符号,它表示跳过该网格的命名,在使用时也不会占用这个位置

9.3.4 间隔

`gap` 属性是用来设置网格行与列之间的间隙 (gutters), 该属性是 `row-gap` and `column-gap` 的简写形式。

9.5 小游戏

通常我们在布局的时候,在控制页面整体布局,版心等时候,我们会优先选择栅格布局,而在小型布局场景如指定一个个人信息卡片时,用flex会更加的高效方便。

在本节的最后强烈推荐3个CSS小游戏来巩固一下 `flex` 和 `grid` 的使用

- <https://flexboxfroggy.com/>
- <http://www.flexboxdefense.com/>
- <https://cssgridgarden.com/>

10. 函数与变量

CSS的值类型中可以使用函数表示更复杂的数据类型或调用特殊的数据处理或计算, 本节不讲述与过渡和动画相关的函数, 将全放入相关章节进行讲解。

10.1 计算

`min()` 方法拥有一个或多个逗号分隔符表达式作为参数, 表达式的值中最小的值作为参数值。表达式可以是数学函数, 字面量, 或者其他表达式, 可以求得有效值的类型。如果你愿意, 你甚至可以在表达式中给每个值一个不同的单位。并且在需要的地方只用圆括号改变计算优先级。

`max()` 方法接受一个或多个用逗号分隔的表达式作为他的参数, 数值最大的表达式的值将会作为指定的属性的值, 其他与 `min()` 类似。

使用举例

```
height: 100%;
max-height: 100px;
=>
height: min(100%, 100px)

height: 100%;
min-height: 100px;
=>
height: max(100%, 100px)
```

`calc()` 函数用一个表达式作为它的参数, 用这个表达式的结果作为值。这个表达式可以是任何如下操作符的组合, 采用标准操作符处理法则的简单表达式。

```
calc(100px + 100vw - 5rem)
```

需要注意的是在使用乘法除法的时候, 需要让表达式有意义, 比如除法的**右边需要为数字**, 乘法需要**至少一边为数字**, 加和减法运算时**两边必须有空白字符**。

`clamp()` 函数接收三个用逗号分隔的表达式作为参数, 按最小值、首选值、最大值的顺序排列。

- 当首选值比最小值要小时, 则使用最小值。
- 当首选值介于最小值和最大值之间时, 用首选值。
- 当首选值比最大值要大时, 则使用最大值。

这个表达式可以是数学函数、字面量或其它计算为有效的参数类型 表达式, 或嵌套的 `min` 和 `max`。作为数学表达式, 你可以使用加减乘除运算而无需使用 `calc()` 函数。你也可以用括号来确定计算顺序。

`clamp(MIN, VAL, MAX)` 其实就是表示 `max(MIN, min(VAL, MAX))`

10.2 变量

`attr()` 用来获取选择到的元素的某一HTML属性值, 并用于其样式。它也可以用于伪元素, 属性值采用伪元素所依附的元素。

`data-xxx` 是一个规范性而不是强制的写法

```

<style>
  a::after {
    content: '链接地址 ' attr(data-url);
  }
</style>
<section>
  <a href="" data-url="https://www.baidu.com"></a>
</section>

```

结果如下

链接地址 <https://www.baidu.com>

`var()` 函数可以代替元素中任何属性中的值的任何部分。`var()` 函数不能作为属性名、选择器或者其他除了属性值之外的值。第一个参数是要替换的自定义属性的名称。函数的可选第二个参数用作回退值。如果第一个参数引用的自定义属性无效，则该函数将使用第二个值。

我们在上面的例子加入一些属性

```

:root {
  --bg-r: red;
  --bg-b: blue;
  --bg-g: green;
  --txt-w: white;
}

a::after {
  content: '链接地址 ' attr(data-url);
  background-color: var(--bg-r);
  color: var(--txt-w);
}

```

我们在根元素下声明了四个变量并在链接中通过 `var()` 使用他们

链接地址 <https://www.baidu.com>

这个特性在各种UI框架中见过不少，大家有兴趣可以翻翻看。

10.3 滤镜

`filter` CSS属性将模糊或颜色偏移等图形效果应用于元素。滤镜通常用于调整图像，背景和边框的渲染。类似photoshop中的滤镜，CSS为我们提供了很多图像处理的滤镜函数。这里不详细列举，可以去MDN上查看详情 [filter](#)

11. @规则

在学习和使用CSS的过程中我们常常能看见CSS顶部或底部以@符号开头的语句，这里定义了一些规则供我们后续CSS的使用，本节将对一些我们常见的规则进行认识。

11.1 @charset

`@charset` 指定样式表中使用的字符编码。它必须是样式表中的第一个元素，而前面不得有任何字符。因为它不是一个嵌套语句，所以不能在@规则条件组中使用。如果有多个 `@charset` @规则被声明，只有第一个会被使用，而且不能在HTML元素或HTML页面的字符集相关 `style` 元素内的样式属性内使用。

常见用法

```
@charset "UTF-8";
@charset "utf-8"; /*大小写不敏感*/
/* 设置css的编码格式为Unicode UTF-8 */
@charset 'iso-8859-15'; /* 无效的，使用了错误的引号 */
@charset 'UTF-8';      /* 无效的，使用了错误的引号 */
@charset "UTF-8";      /* 无效的，多于一个空格 */
    @charset "UTF-8";   /* 无效的，在at-rule之前多了一个空格 */
@charset UTF-8;        /* 无效，它不是一个有效的字符串 */
```

11.2 @font-face

`@font-face` 指定一个用于显示文本的自定义字体；字体能从远程服务器或者用户本地安装的字体加载。如果提供了 `local()` 函数，从用户本地查找指定的字体名称，并且找到了一个匹配项，本地字体就会被使用。否则，字体就会使用 `url()` 函数下载的资源。

基本使用

```
@font-face {
  font-family: "Open Sans";
  src: url("/fonts/OpenSans-Regular-webfont.woff2") format("woff2"),
       url("/fonts/OpenSans-Regular-webfont.woff") format("woff");
}
```

11.3 @media

`@media` 可用于基于一个或多个**媒体查询**的结果来应用样式表的一部分。使用它，您可以指定一个媒体查询和一个CSS块，当且仅当该媒体查询与正在使用其内容的设备匹配时，该CSS块才能应用于该文档。

媒体查询常被用于以下目的：

- 有条件的通过 `@media` 和 `@import` CSS 装饰样式。
- 用 `media=` 属性为 `style`, `link`, `source` 和其他HTML元素指定特定的媒体类型。如：

```
<link rel="stylesheet" src="styles.css" media="screen" />
<link rel="stylesheet" src="styles.css" media="print" />
```

媒体类型（Media types）描述设备的一般类别。除非使用 `not` 或 `only` 逻辑操作符，媒体类型是可选的，并且会（隐式地）应用 `all` 类型。

值	说明
<code>all</code>	适用于所有设备。
<code>print</code>	适用于在打印预览模式下在屏幕上查看的分页材料和文档
<code>screen</code>	主要用于屏幕。
<code>speech</code>	主要用于语音合成器。

逻辑操作符:

值	说明
<code>and</code>	用于将多个媒体查询规则组合成单条媒体查询，当每个查询规则都为真时则该条媒体查询为真，它还用于将媒体功能与媒体类型结合在一起。
<code>not</code>	用于否定媒体查询，如果不满足这个条件则返回true，否则返回false。如果出现在以逗号分隔的查询列表中，它将仅否定应用了该查询的特定查询。如果使用not运算符，则还必须指定媒体类型。
<code>only</code>	<code>only</code> 运算符仅在整个查询匹配时才用于应用样式 如果使用 <code>only</code> 运算符，则还必须指定媒体类型。
<code>,</code>	逗号用于将多个媒体查询合并为一个规则。逗号分隔列表中的每个查询都与其他查询分开处理。因此，如果列表中的任何查询为true，则整个media语句均返回true。换句话说，列表的行为类似于逻辑或 <code>or</code> 运算符。

```
@media print { ... }; /* 当前设备为打印机时生效 */
@media screen, print { ... }; /* 当前设备为屏幕或打印机生效 */
@media (hover: hover) { ... }; /* 当输入设备可以触发hover时生效 */
@media (max-width: 12450px) { ... }; /* 当屏幕不超过12450px宽时生效 */
@media (color) { ... } /* 当设备带有色彩时生效 */
@media speech and (aspect-ratio: 11/5) { ... } /* 当设备为语音合成器且屏幕宽高比为11比5时生效 */
@media (min-width: 30em) and (orientation: landscape) { ... } /* 当设备最小30em宽且为横屏设备时生效 */
```

11.4 @keyframes

关键帧 `@keyframes` 通过在动画序列中定义关键帧的样式来控制CSS动画序列中的中间步骤。和 `transition` 相比，关键帧 `keyframes` 可以控制动画序列的中间步骤。

具体用法在动画部分说明。

11.5 @import

`@import` 用于从其他样式表导入样式规则。这些规则**必须先于所有其他类型的规则**，`@charset` 规则除外；

基本用法 `@import url`，`url` 表示要引入资源位置的字符串或者可以使用 `url()` 函数

```
@import 'custom.css';
@import url("chrome://communicator/skin/");
```

12. 过渡与动画

过渡和动画是实现页面交互和特效的重中之重，本节将展开对这两个特性的回顾。

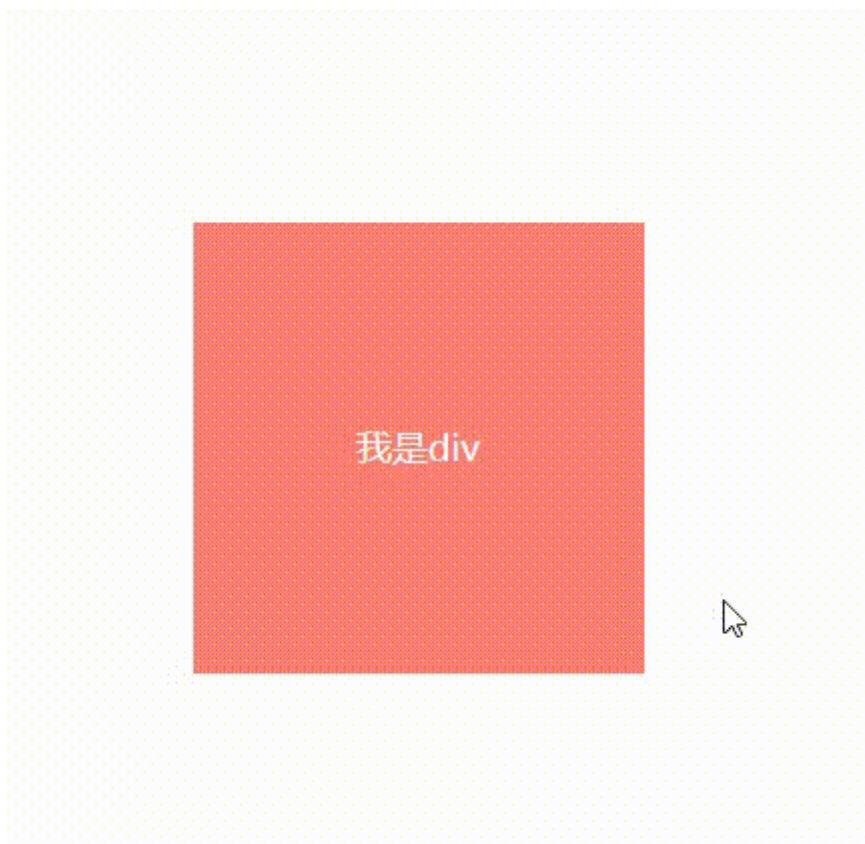
12.1 过渡

CSS transitions 提供了一种在更改CSS属性时控制动画速度的方法。其可以让属性变化成为一个持续一段时间的过程，而不是立即生效的。比如，将一个元素的颜色从白色改为黑色，通常这个改变是立即生效的，使用 CSS transitions 后该元素的颜色将逐渐从白色变为黑色，按照一定的曲线速率变化。这个过程可以自定义。

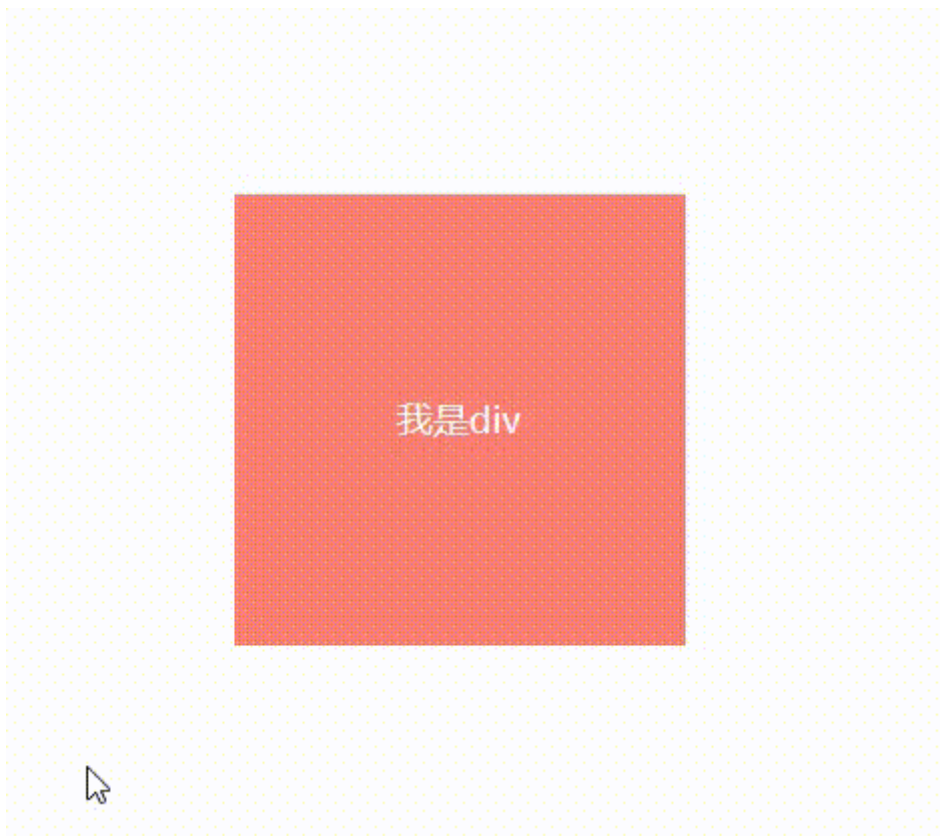
通常将两个状态之间的过渡称为**隐式过渡 (implicit transitions)**，因为开始与结束之间的状态由浏览器决定。

话不多说，先体验一波

过渡前：



下面是添加过渡后的，明显就感觉丝滑了许多



12.1.1 基本属性

值	说明
<code>transition-property</code>	指定哪个或哪些 CSS 属性用于过渡。只有指定的属性才会在过渡中发生动画，其它属性仍如通常那样瞬间变化。
<code>transition-duration</code>	指定过渡的时长。或者为所有属性指定一个值，或者指定多个值，为每个属性指定不同的时长。
<code>transition-delay</code>	指定延迟，即属性开始变化时与过渡开始发生时之间的时长。

12.2.2 过渡曲线

CSS属性受到 过渡效果的影响，会产生不断变化的中间值，而 `transition-timing-function` 属性用来描述这个中间值是怎样计算的。实质上，通过这个函数会建立一条加速度曲线，因此在整个transition变化过程中，变化速度可以不断改变。

值	描述
<code>linear</code>	规定以相同速度开始至结束的过渡效果（等于 cubic-bezier(0,0,1,1)）。
<code>ease</code>	规定慢速开始，然后变快，然后慢速结束的过渡效果（cubic-bezier(0.25,0.1,0.25,1)）。
<code>ease-in</code>	规定以慢速开始的过渡效果（等于 cubic-bezier(0.42,0,1,1)）。
<code>ease-out</code>	规定以慢速结束的过渡效果（等于 cubic-bezier(0,0,0.58,1)）。
<code>ease-in-out</code>	规定以慢速开始和结束的过渡效果（等于 cubic-bezier(0.42,0,0.58,1)）。
<code>cubic-bezier(n,n,n,n)</code>	在 <code>cubic-bezier</code> 函数中定义自己的值。可能的值是 0 至 1 之间的数值。

简写：

transition: 变化属性 过渡时长 过渡曲线 过渡延时； /* 若变化属性有多个时用逗号分割开来 */

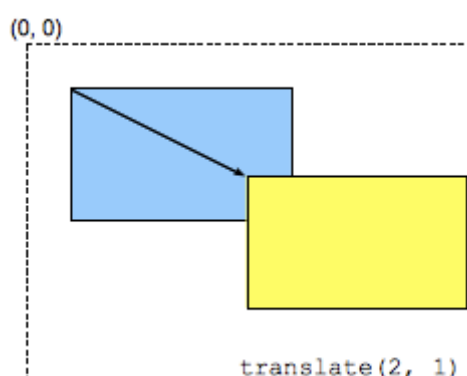
CSS 过渡 由简写属性 `transition` 定义是最好的方式，可以避免属性值列表长度不一，节省调试时间。

12.2 动画

CSS transforms 通过一系列 CSS 属性实现，通过使用这些属性，可以对 HTML 元素进行线性仿射变形（affine linear transformations）。可以进行的变形包括旋转，倾斜，缩放以及位移，这些变形同时适用于平面与三维空间。

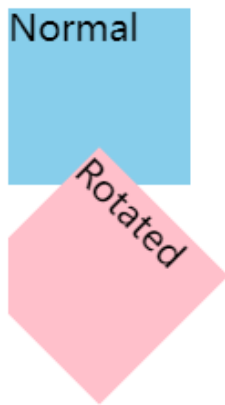
12.2.1 变换函数(2D)

`translate()` 水平和/或垂直方向上重新定位元素。它是 `translateX()`和`translateY()` 的简写



它可以接受两个任意的长度单位，当接受百分比时，是以变换元素**自身**的宽高为参照的。

`rotate()` 函数定义了一种将元素围绕一个定点（由 `transform-origin` 属性指定）**旋转而不变形**的转换。指定的角度定义了旋转的量度。若角度为**正**，则顺时针方向旋转，否则**逆时针**方向旋转。元素旋转的固定点 - 如上所述 - 也称为**变换原点**。这默认为元素的中心，但你可以使用 `transform-origin` 属性设置自己的自定义变换原点。



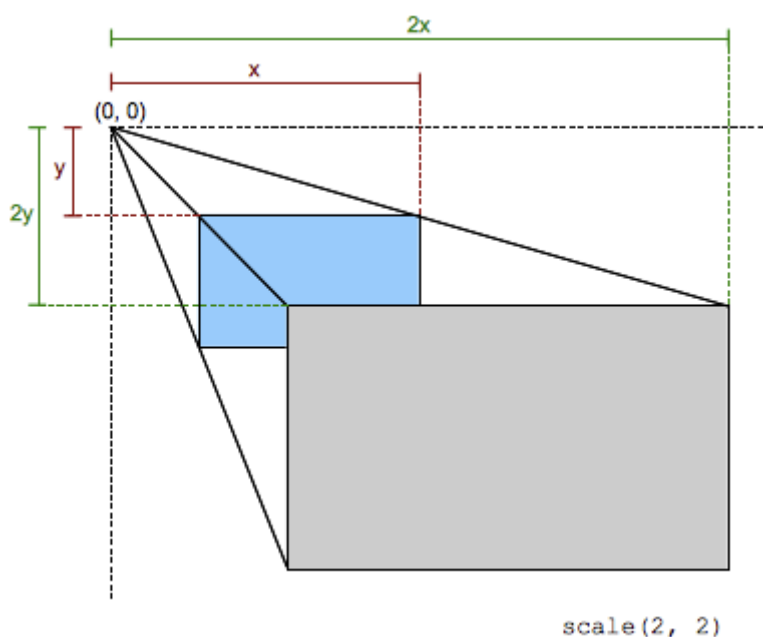
在二维平面上 `rotate()` 的表现与 `rotateZ()` 是一致的

`transform-origin` 更改一个元素变形的原点，默认的转换原点是 `center`。可以使用一个，两个或三个值来指定，其中每个值都表示一个偏移量。

- 一个值：
 - 必须是长度，百分比，或 `left`, `center`, `right`, `top`, `bottom` 关键字中的一个。
- 两个值：
 - 其中一个必须是长度，百分比，或 `left`, `center`, `right` 关键字中的一个。
 - 另一个必须是长度，百分比，或 `top`, `center`, `bottom` 关键字中的一个。
- 三个值：
 - 前两个值和只有两个值时的用法相同。
 - 第三个值必须是长度。它始终代表Z轴偏移量。

当使用**非关键字**时的参照为左侧或顶部。

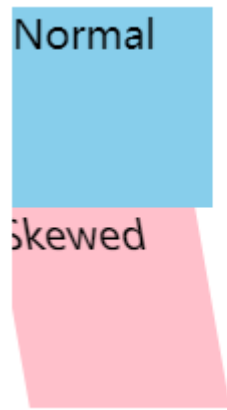
`scale()` 用于修改元素的大小。可以通过向量形式定义的缩放值来放大或缩小元素，同时可以在不同的方向设置不同的缩放值。



若传入一个值则**等比放大**，传入**两个值且不相等**为分别在x和y轴上放大

`skew()` 函数定义了一个元素在二维平面上的倾斜转换，它是 `skewX()` 和 `skewY()` 的缩写。这种转换是一种剪切映射(横切)，它在水平和垂直方向上将单元内的每个点扭曲一定的角度。每个点的坐标根据指定的角度以及到原点的距离，进行成比例的值调整；因此，一个点离原点越远，其增加的值就越大。

传入一个值时相当于 `skewX()` 表示用于沿横坐标扭曲元素的角度。



传入两个值分别代表横轴与纵轴扭曲的角度。



12.2.2 透视

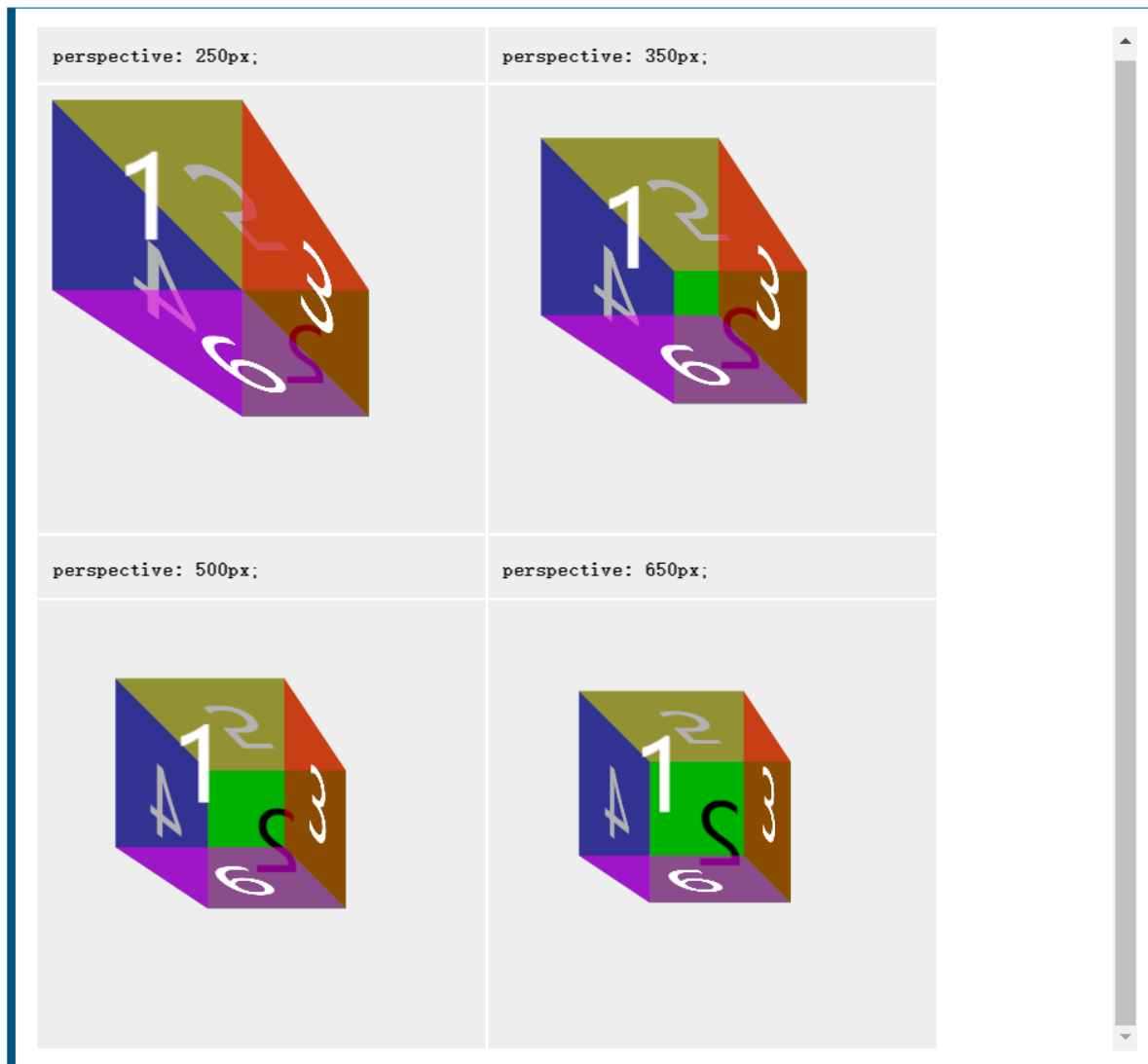
在看3D的变换函数之前，我们先来看一下 `perspective` 属性的用法。

`perspective` 指定了观察者与 $z=0$ 平面的距离，使具有三维位置变换的元素产生透视效果。 $z>0$ 的三维元素比正常大，而 $z<0$ 时则比正常小，大小程度由该属性的值决定。举个通俗的例子，“横看成岭侧成峰，远近高低各不同”，生活中处处充满着这种现象，当我们在倾斜角度相同的境况下，我们离物体的远近不同，所看到的是不一样的，接下来引用一个MDN的例子加深一下理解。

设置视角

此示例显示了一个立方体，其 `perspective` 设置为不同的值。立方体的收缩由 `perspective` 属性定义。它的值越小，视角越深。

Result



三维元素在观察者后面的部分不会绘制出来，即 `z` 轴坐标值大于 `perspective` 属性值的部分。默认情况下，消失点位于元素的中心，但是可以通过设置 `perspective-origin` 属性来改变其位置。

`perspective-origin` 指定了观察者的位置，用作 `perspective` 属性的消失点。

x-position

指定消失点的横坐标，其值有以下形式：

- 长度值或相对于元素宽度的百分比值，可为负值。
- `left`，关键字，0值的简记。
- `center`，关键字，50%的简记。
- `right`，关键字，100%的简记。

y-position

指定消失点的纵坐标，其值有以下形式：

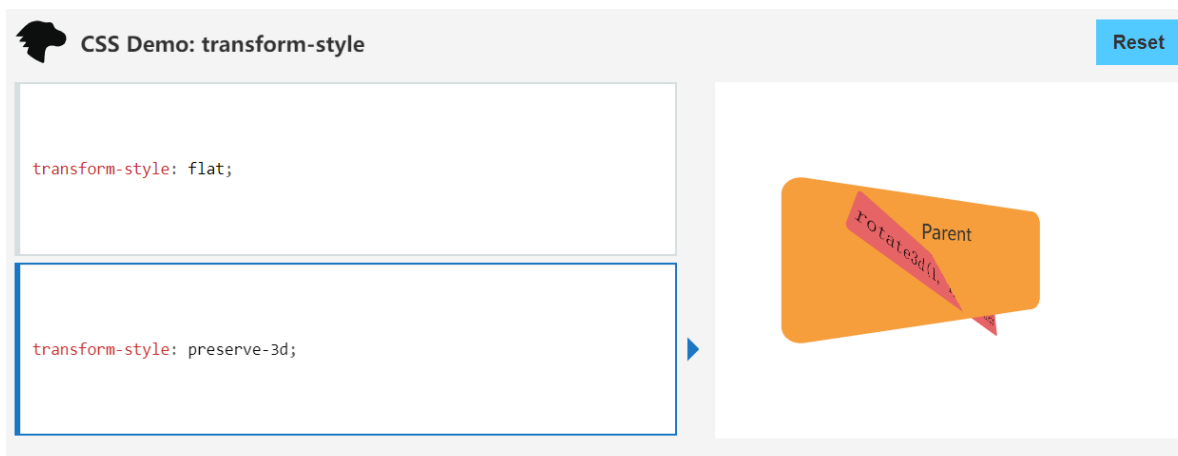
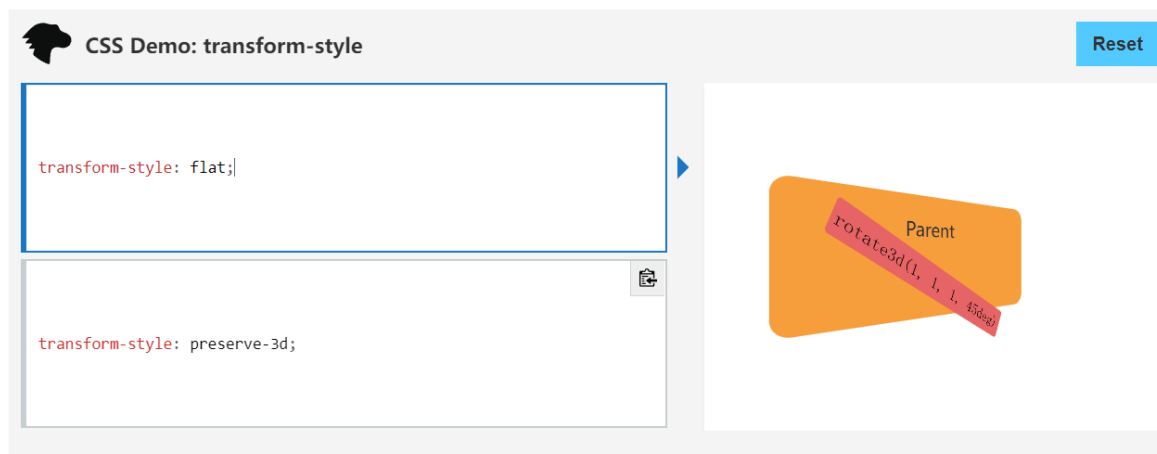
- 长度值或相对于元素高度的百分比值，可为负值。

- `top`, 关键字, 0值的简记。
- `center`, 关键字, 50%的简记。
- `bottom`, 关键字, 100%的简记。

注意: `perspective` 用于单独属性时是对内部元素整体透视, 而在 `transform` 中作为函数使用则为对单个元素进行透视而不和其他同一层级的元素作为统一平面进行透视。

12.2.3 变换函数(3D)

`transform-style` 设置元素的子元素是位于 3D 空间中还是平面中。 `flat` 设置元素的子元素位于该元素的平面中。 `preserve-3d` 指示元素的子元素应位于 3D 空间中, 如果选择平面, 元素的子元素将不会有 3D 的遮挡关系。



接下来再来看看我们之前跳过的几个3D属性

`rotate3d()` 定义一个变换, 它将元素围绕固定轴移动而不使其变形。运动量由指定的角度定义; 如果为正, 运动将为顺时针, 如果为负, 则为逆时针。它接受4个值分别为 `[x, y, z]` 和旋转角度, 前者为一组3D的方向向量描述旋转轴, 后者定义旋转的角度。

与平面旋转相反的是, 3D旋转的组合通常是不可交换的; 这意味着定义旋转规则的值的顺序是严格控制的。

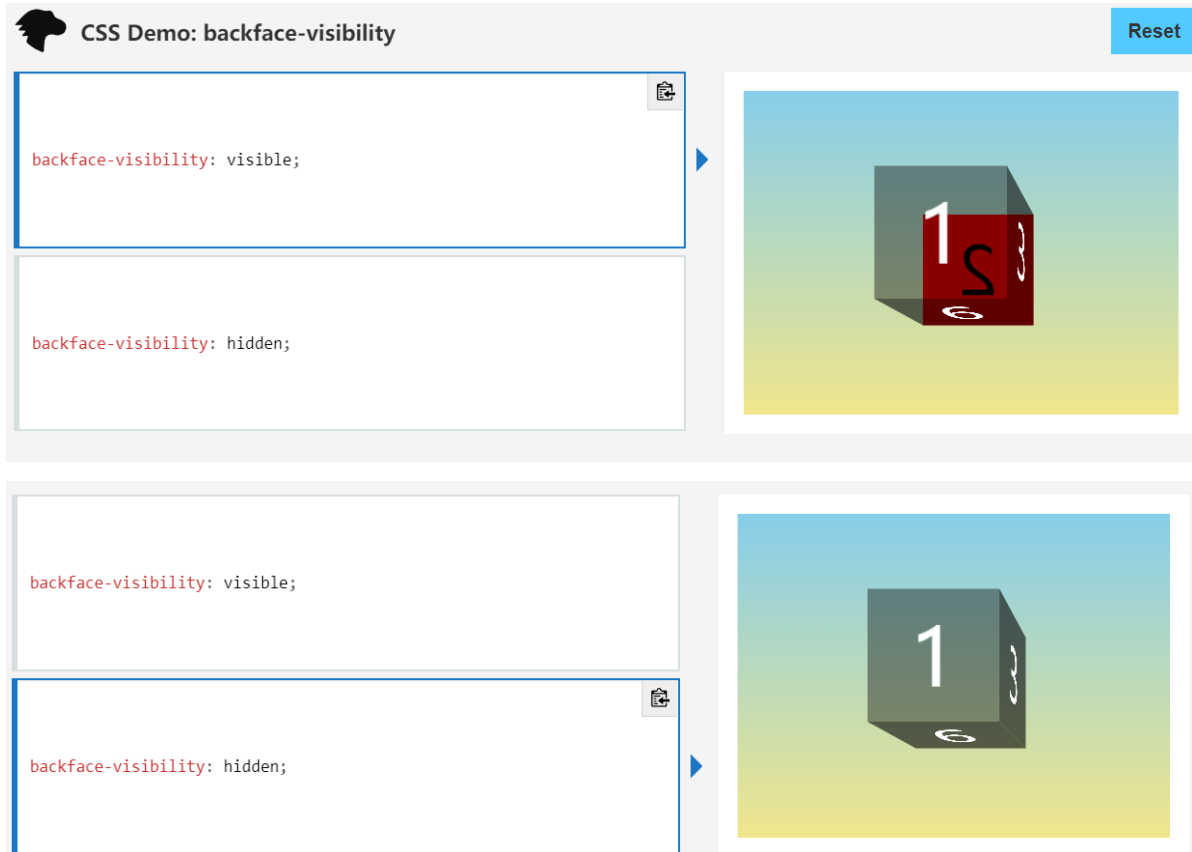
`translate3d()` 在3D空间内移动一个元素的位置。这个移动由一个三维向量来表达, 分别表示他在三个方向上移动的距离。

与2D相比多了一个Z轴的偏移, 需要注意的是, Z轴取值百分比是没有意义的。

`scale3d()` 用法与在2D中使用一致, 增加了一个Z轴的缩放。

`backface-visibility` 指定当元素背面朝向观察者时是否可见, 有 `visible` 和 `hidden` 两个属性来表示显示与否。

示例:



12.2.4 帧动画

定义动画: 通过使用 `@keyframes` 建立两个或两个以上关键帧来实现。每一个关键帧都描述了动画元素在给定的时间点上应该如何渲染。关键帧使用百分比来指定动画发生的时间点。0%表示动画的第一时刻，100%表示动画的最终时刻。因为这两个时间点十分重要，所以还有特殊的别名：`from`和`to`。这两个都是可选的，若`from/0%`或`to/100%`未指定，则浏览器使用计算值开始或结束动画。

示例:

```
@keyframes slidein {  
  from {  
    margin-left: 100%;  
    width: 300%;  
  }  
  
  to {  
    margin-left: 0%;  
    width: 100%;  
  }  
}
```

配置动画: 创建动画序列，需要使用 `animation` 属性或其子属性，该属性允许配置动画时间、时长以及其他动画细节

基本属性:

值	说明
<code>animation-name</code>	指定应用的一系列动画，每个名称代表一个由 <code>@keyframes</code> 定义的动画序列。
<code>animation-duration</code>	指定一个动画周期的时长
<code>animation-delay</code>	定义动画于何时开始，即从动画应用在元素上到动画开始的这段时间的长度,默认为0s

`animation-timing-function` 属性定义CSS动画在每一动画周期中执行的节奏,用法与 `transition-timing-function` 类似。

`animation-iteration-count` 定义动画在结束前运行的次数 可以是1次 无限循环，值为数字的时候代表动画执行的次数，当使用关键字 `infinite` 的时候代表无限循环。

`animation-direction` CSS 属性指示动画是否反向播放：

值	说明
<code>normal</code>	每个循环内动画向前循环，换言之，每个动画循环结束，动画重置到起点重新开始，这是默认属性。
<code>alternate</code>	动画交替反向运行，反向运行时，动画按步后退，同时，带时间功能的函数也反向，比如， <code>ease-in</code> 在反向时成为 <code>ease-out</code> 。计数取决于开始时是奇数迭代还是偶数迭代。
<code>reverse</code>	反向运行动画，每周期结束动画由尾到头运行。
<code>alternate-reverse</code>	反向交替，反向开始交替，动画第一次运行时是反向的，然后下一次是正向，后面依次循环。决定奇数次或偶数次的计数从1开始。

`animation-fill-mode` 设置CSS动画在执行之前和之后如何将样式应用于其目标。

值	说明
<code>none</code>	当动画未执行时，动画将不会将任何样式应用于目标，而是已经赋予给该元素的CSS 规则来显示该元素。这是默认值。
<code>forwards</code>	目标将保留由执行期间遇到的最后一个关键帧计算值。
<code>backwards</code>	动画将在应用于目标时立即应用第一个关键帧中定义的值，并在 <code>animation-delay</code> 期间保留此值。
<code>both</code>	动画将遵循 <code>forwards</code> 和 <code>backwards</code> 的规则，从而在两个方向上扩展动画属性。

`animation-play-state` 定义一个动画是否运行或者暂停。可以通过查询它来确定动画是否正在运行。另外，它的值可以被设置为暂停和恢复的动画的重放。恢复一个已暂停的动画，将从它开始暂停的时候，而不是从动画序列的起点开始在动画。

- `running` 当前动画正在运行。
- `paused` 当前动画已被停止。

13. 表格

CSS Table 是一个定义如何展示表格数据的CSS模块，在前端数据展示中有很大的用处。

表格的很多HTML属性现在都已弃用，最好使用CSS来代替他们。具体信息可查看[table标签](#)。

在了解具体用法之前，我们先来看看 `display` 中有哪些表格相关的属性。

13.1 display中的表格

值	属性
<code>table-row</code>	表现与 <code>tr</code> 标签一致
<code>table-row-group</code>	表现与 <code>tbody</code> 标签一致
<code>table-footer-group</code>	表现与 <code>tfoot</code> 标签一致
<code>table-header-group</code>	表现与 <code>thead</code> 标签一致
<code>table-cell</code>	表现与 <code>td</code> 标签一致
<code>table-caption</code>	表现与 <code>caption</code> 标签一致

13.2 表格的基本属性

`caption-side` 会将表格的标题 `caption` 标签 放到规定的位置。

- `top` 标题出现在表格的上方
- `bottom` 标题出现在表格的下方

`empty-cells` 定义了用户端 user agent 应该怎么来渲染表格中没有可见内容的单元格的边框和背景。

- `show` 边框和背景像普通元素一样正常渲染
- `hide` 边框和背景被隐藏

`table-layout` 定义了用于布局表格单元格，行和列的算法。

- `auto` 大多数浏览器采用自动表格布局算法对表格布局，表格及单元格的宽度取决于其包含的内容。
- `fixed` 表格和列的宽度通过表格的宽度来设置，某一列的宽度仅由该列首行的单元格决定。在当前列中，该单元格所在行之后的行并不会影响整个列宽。

`border-collapse` 是用来决定表格的边框是分开的还是合并的。在分隔模式下，相邻的单元格都拥有独立的边框。在合并模式下，相邻单元格共享边框。

`border-spacing` 属性指定相邻单元格边框之间的距离。

这两个属性在我们介绍边框的时候已经提过了[传送门](#)。

`vertical-align` 用来指定行内元素或表格单元格（`table-cell`）元素的垂直对齐方式。

这个属性在回答垂直居中的题目时可以说一说，在行元素和行内块元素中常用来对齐图片和文本，在 `table-cell` 中可以用来垂直居中。

14. 总结

感觉CSS基础大部分考点都在上面了，如果平时经常写CSS的话是无需刻意去背的。但总有需要查缺补漏的地方，基础扎实点总是好的。面试例举的并不是很多，因为大部分都在知识点里面已经有答案了，所以主要列举了几个高频的考题和一些小案例。另外，如CSS性能优化，使用规范等个人感觉更应该放在浏览器原理、前端工程化等部分复习，其实就是偷懒不想写字了。

